

**A REPORT
ON

SEMANTIC RECONSTRUCTION OF SOFTWARE INTENT: A CLOUD-HOSTED
AGENTIC AI SOLUTION FOR STUDY OF REQUIREMENT TRACEABILITY IN
PRODUCTION GRADE CODEBASES**

BY

Aarya Nanndaann Singh M N

ID No.: 2024MT03013

AT

Aqueralabs India Pvt Ltd, Bangalore - 560066

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

VIDYA VIHAR, PILANI, RAJASTHAN - 333031

May 2026

**A REPORT
ON
SEMANTIC RECONSTRUCTION OF SOFTWARE INTENT: A CLOUD-HOSTED
AGENTIC AI SOLUTION FOR STUDY OF REQUIREMENT TRACEABILITY IN
PRODUCTION GRADE CODEBASES**

BY

Aarya Nanndaann Singh M N

ID No.: 2024MT03013

Discipline: M.Tech in Cloud Computing

WILP Course No.: CCZG628T - Dissertation

*Prepared in full fulfilment of the
WILP Dissertation / Project / Project Work Course*

AT

Aqueralabs India Pvt Ltd, Bangalore - 560066

Under the Supervision of

Mr. Hanumanthu Indrakanti (Director of Engineering)

Aqueralabs India Pvt Ltd, Bangalore

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

VIDYA VIHAR, PILANI, RAJASTHAN - 333031

May 2026

CERTIFICATE FROM THE SUPERVISOR

This is to certify that the dissertation entitled "Semantic Reconstruction of Software Intent: A Cloud-Hosted Agentic AI Solution for Study of Requirement Traceability in Production Grade Codebases", submitted by Mr. Aarya Nanndaann Singh M N (ID No.: 2024MT03013), in full fulfilment of the requirements of the M.Tech (Cloud Computing) WILP Dissertation Course (CCZG628T) of the Birla Institute of Technology and Science, Pilani, is a bona fide record of the original work carried out by him under my supervision and guidance at Aqueralabs India Pvt Ltd, Bangalore.

The dissertation has been completed in accordance with the academic standards and timelines set by the WILP Division. The work embodied in the report has not been submitted, in part or in full, for the award of any other degree or diploma in this or any other institute. To the best of my knowledge and belief, the contents of the report represent the candidate's independent technical and research effort and have been carried out in a manner suitable for a Master's level dissertation in Cloud Computing and Artificial Intelligence. I recommend the report for the consideration of the panel of examiners and the WILP Division.



Signature of the Supervisor

Name: Mr. Hanumanthu Indrakanti

Designation: Director of Engineering

Organization: Aqueralabs India Pvt Ltd,
Bangalore - 560066

Date: 09 May 2026

Place: Bangalore



Signature of the Additional Examiner

Name: Mr. Deepan Kumar

Designation: Staff Engineer

Organization: Aqueralabs India Pvt Ltd,
Bangalore - 560066

Date: 09 May 2026

Place: Bangalore

ACKNOWLEDGEMENTS

The work presented in this dissertation has been shaped by the guidance, encouragement and support of several individuals, and it is my pleasure to acknowledge them here. I am sincerely grateful to the leadership of Aqueralabs India Pvt Ltd, Bangalore, for providing me with the freedom and the production engineering environment in which the ideas behind *IntentEra* could be framed, prototyped and validated. The exposure to real production code repositories and operational developer workflows has been central to the relevance of this work.

I owe a particular debt of gratitude to my supervisor, Mr. Hanumanthu Indrakanti, Director of Engineering, for his consistent technical guidance, patience during architectural discussions and insistence that the work remain practically deployable rather than purely theoretical. His feedback significantly influenced the design of the multi-agent reasoning layer and the cloud topology towards production-grade quality.

I am equally thankful to Mr. Deepan Kumar, Staff Engineer, for his rigorous review of the system design and his valuable suggestions on the retrieval-augmented generation pipeline, which strengthened the methodological foundations of this work.

I would also like to express my sincere gratitude to Dr. Nishit Narag, Head of Department, and to my Faculty Mentor and Evaluator, Mr. Paresh Saxena, for their academic guidance, encouragement and support throughout the dissertation process. Their feedback and evaluation helped refine both the technical and research dimensions of this work.

I would like to thank the BITS Pilani WILP Division, particularly the Course-in-Charge of the Dissertation course (CCZG628T), for providing a structured academic framework that made it possible to pursue this two-semester dissertation alongside a full-time engineering role. The review process and evaluation methodology were instrumental in maintaining direction and consistency throughout the research.

Finally, I extend my appreciation to my colleagues at Aqueralabs for their support, collaboration and constructive discussions during the course of this work.

Aarya Nanndaann Singh M N

ID No.: 2024MT03013

BITS Pilani - WILP Division

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE, PILANI (RAJASTHAN)

(MAY, 2026)

WILP Division - Abstract Sheet

Organization	Aqueralabs India Pvt Ltd, Bangalore - 560066
Location	Bangalore, Karnataka, India
Duration	20 January 2026 - 12 May 2026 (one semester)
Date of Start	20 January 2026
Date of Submission	12 May 2026
Title of the Project	Semantic Reconstruction of Software Intent: A Cloud-Hosted Agentic AI Solution for Study of Requirement Traceability in Production Grade Codebases
ID No. / Name of the Student	2024MT03013 / Aarya Nanndaann Singh M N
Supervisor / Additional Examiner	Mr. Hanumanthu Indrakanti (Supervisor) Mr. Deepan Kumar (Additional Examiner)

Faculty Mentor: Paresh Saxena (psaxena@hyderabad.buts-pilani.ac.in)

Project Areas: Cloud Computing; Artificial Intelligence; Software Engineering;
Developer Tools and Productivity; Information Retrieval.

Keywords: Requirement Traceability; Software Intent Reconstruction; Retrieval-
Augmented Generation; Agentic AI; Multi-Agent Systems; Large Language Models;
AWS Lambda; MongoDB Atlas Vector Search; Developer Productivity; Code
Comprehension.

Abstract:

Modern enterprise codebases evolve through fast, distributed DevOps workflows in which the rationale behind code changes is fragmented across issue trackers, commits, pull requests and informal discussions. Source code captures what a system does, but rarely preserves why each change was introduced. This dissertation presents IntentEra, a cloud-hosted agentic AI framework that reconstructs software intent and establishes requirement traceability in production-grade codebases. IntentEra ingests Jira, Confluence and GitHub artifacts on AWS Lambda, normalises and section-aware chunks them, generates 1536-dimensional embeddings with OpenAI and indexes them in two MongoDB Atlas Vector Search collections. A multi-query Retrieval-Augmented Generation pipeline fans out the user's question and the local commit history into both stores in parallel, max-pools the results by chunk identifier and asks an LLM, constrained to JSON-mode, to compose a non-technical “why” explanation with grounded citations. A VS Code and Cursor sidebar extension brings the reconstructed intent directly into the developer's workflow. The system is deployed end-to-end on AWS and demonstrates that intent reconstruction can be performed reliably, incrementally and at low cost on real engineering artifacts.

**Signature of Student**

Name: Aarya Nanndaann Singh M N

Date: 09 May 2026

**Signature of Supervisor**

Name: Mr. Hanumanthu Indrakanti

Date: 09 May 2026

LIST OF ABBREVIATIONS AND ACRONYMS

ADF	Atlassian Document Format
ANN	Approximate Nearest Neighbour
API	Application Programming Interface
AWS	Amazon Web Services
BITS	Birla Institute of Technology and Science
CI / CD	Continuous Integration / Continuous Delivery
CLI	Command-Line Interface
CRUD	Create, Read, Update, Delete
DevOps	Development and Operations
GraphQL	Graph Query Language
HTTP / HTTPS	Hypertext Transfer Protocol (Secure)
IAM	Identity and Access Management
IDE	Integrated Development Environment
IR	Information Retrieval
ITSM	IT Service Management
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
NLP	Natural Language Processing
PAT	Personal Access Token
PDF / DOCX /	Portable Document Format / Office Open XML Document /

HTML	HyperText Markup Language
PR	Pull Request
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SHA	Secure Hash Algorithm
SSO	Single Sign-On
TLS	Transport Layer Security
TLX	Task Load Index (e.g. NASA-TLX)
URI / URL	Uniform Resource Identifier / Locator
VPC	Virtual Private Cloud
VS Code	Visual Studio Code
VSIX	Visual Studio Extension Installer
WILP	Work-Integrated Learning Programmes

TABLE OF CONTENTS

Chapter 1 - Introduction.....	1
1.1 Background and Motivation.....	1
1.2 Problem Setting in the Employing Organization	2
1.3 Statement of the Problem.....	2
1.4 Scope and Approach.....	3
1.5 Summary of Contributions.....	3
1.6 Organisation of the Report	4
Chapter 2 - Literature Survey.....	6
2.1 Natural-Language Processing for Requirement Traceability	6
2.2 Empirical Comparison of Requirement-to-Code Link Recovery.....	7
2.3 Retrieval-Augmented Generation for Traceability	7
2.4 Transformer Models for Code	8
2.5 Agentic AI for Software Engineering.....	9
2.6 Positioning and Gap	9
Chapter 3 - Problem Statement, Objectives and Scope	11
3.1 Formal Problem Statement	11
3.2 Dissertation Objectives	11
3.3 Scope of Work.....	12
3.4 Success Criteria	13

3.5 Plan of Work	13
Chapter 4 - System Architecture.....	16
4.1 Architectural Principles.....	16
4.2 The Five Layers.....	16
4.2.1 Artifact Ingestion Layer	16
4.2.2 Knowledge Extraction Layer	17
4.2.3 Semantic Reasoning Engine.....	17
4.2.4 Agent Orchestration Layer.....	18
4.2.5 IDE Integration Layer	18
4.3 Cross-Cutting Services	18
4.4 Significance of the Design	19
Chapter 5 - Cloud-Hosted Implementation on AWS.....	20
5.1 Compute: Three AWS Lambda Functions	20
5.2 Lambda Environment Variables	23
5.3 Scheduling: Amazon EventBridge.....	25
5.4 Credentials: AWS Secrets Manager	28
5.5 Front Door: Amazon API Gateway.....	29
5.6 Vector Store: MongoDB Atlas Vector Search	32
5.7 Sync State: Upstash Redis.....	34
5.8 Observability: CloudWatch Logs	35

5.9 Identity and Access Management	36
5.10 Operating Cost	37
5.11 Region and Availability Choices	38
Chapter 6 - Multi-Source RAG and Multi-Agent Reasoning.....	39
6.1 Section-Aware Chunking	39
6.2 Idempotent Upsert	40
6.3 Embedding	40
6.4 The Multi-Query Retriever	41
6.5 LLM Answer Composition with Grounding.....	42
6.6 Multi-Agent Orchestration	43
6.7 Cost and Latency Profile.....	43
Chapter 7 - VS Code and Cursor Chat Extension	45
7.1 The Three Pieces.....	45
7.2 Privacy and Trust Boundary.....	46
7.3 End-to-End Sequence of a Chat Request	46
7.4 The Extension in Action	47
7.5 Configuration Surface.....	48
7.6 Graceful Degradation	49
Chapter 8 - Evaluation and Observations	50
8.1 Functional Verification.....	50

8.2 Retrieval Quality.....	51
8.3 Latency and Cost.....	52
8.4 Qualitative Impact on Developer Cognitive Load	52
8.5 Observations.....	53
Chapter 9 - Challenges and Limitations	55
9.1 Linking Natural-Language Requirements to Code	55
9.2 Noisy and Inconsistent Development Artifacts.....	55
9.3 Incomplete or Missing Traceability.....	55
9.4 Heterogeneity of Data Sources.....	56
9.5 LLM Hallucination	56
9.6 Cold Start and Container Reuse.....	56
9.7 Limitations Acknowledged in Scope	57
Chapter 10 - Conclusion and Future Work.....	58
10.1 Summary of the Work	58
10.2 Contributions Restated	58
10.3 Expected Impact	59
10.4 Future Work.....	59
10.5 Closing Remark.....	60
References	61
Appendix A - Configuration Reference (Excerpt).....	64

Appendix B - Sample Payloads	66
B.1 EventBridge Schedule Payload (GitHub incremental).....	66
B.2 EventBridge Schedule Payload (Jira full import)	66
B.3 POST /retrieve Sample Request.....	66
B.4 POST /chat Sample Request	66
B.5 POST /chat Sample Response (truncated).....	67
B.6 AWS Secrets Manager JSON Skeleton.....	68
Appendix C - CLI Command Reference	70
Glossary	72
Checklist for the Final Dissertation Report.....	75

Chapter 1 - Introduction

1.1 Background and Motivation

Modern enterprise software systems no longer resemble the linear, monolithic codebases that once defined the discipline. They are living ecosystems that evolve through commits, pull requests, incident tickets, requirement updates, design discussions and automated continuous integration pipelines, often spread across geographically distributed teams. The complete picture of what a system is and why it behaves the way it does is therefore not located in a single place; it is fragmented across an entire ecosystem of DevOps artifacts.

In day-to-day practice an engineer is expected not only to understand what a piece of code does, but also why it was written in a particular way, which requirement triggered the change, whether the implementation truly resolves the original defect, and how it interacts with other historical decisions in the codebase. These why-questions are unavoidable during onboarding, defect triage, refactoring and architectural review. They are also the questions that source code by itself cannot answer.

The phenomenon studied in this dissertation is what we call intent erosion: the progressive loss of the contextual rationale behind code as time passes, as authors leave the organisation and as artifacts become stale or scattered. The cost of intent erosion is observable. It manifests as long onboarding ramps, as recurring bugs that re-introduce previously fixed regressions, and as the extra cognitive load of “context archaeology” - the manual, time-consuming work of re-discovering why a system is the way it is by digging through tickets, commits and chat logs.

1.2 Problem Setting in the Employing Organization

The work reported here was carried out at Aqueralabs India Pvt Ltd, an enterprise software product organisation that maintains multiple production-grade backend services. In this environment engineers depend on Atlassian Jira for requirement and incident tracking, on Confluence for design documentation, on GitHub for version control and on a set of CI/CD workflows for build, validation and deployment. While each of these platforms is individually mature, the rationale behind any non-trivial code change is typically split across three or more of them. New joiners and contributors therefore have to perform the same mental joins repeatedly, often with incomplete information.

These observations motivated the search for a system that could act as a semantic intermediary between developers and their fragmented knowledge ecosystem - a system that could ingest and unify the artifacts continuously, reconstruct the intent behind code changes on demand, and surface that reconstructed intent inside the developer's existing tooling rather than as yet another portal to visit.

1.3 Statement of the Problem

The central problem addressed by this dissertation can be stated as follows. Given a production-grade codebase whose history is distributed across an issue tracker (Jira), a code-hosting platform (GitHub) and a documentation platform (Confluence), and given a natural-language question from a developer (typically about a file or a selected range of code), reconstruct and present the implementation intent associated with that code in a manner that is grounded in the underlying artifacts, traceable back to specific commits, tickets and pull requests, and integrated into the developer's IDE.

The problem is challenging because requirements and code live in very different representational spaces, because explicit requirement-to-code links are rarely maintained in practice, and because the volume of artifacts in any non-trivial repository makes manual approaches uneconomical.

1.4 Scope and Approach

The dissertation designs and implements IntentEra, a cloud-hosted agentic AI framework that addresses the above problem through five cooperating layers: (i) artifact ingestion, (ii) knowledge extraction, (iii) a semantic reasoning engine, (iv) an agent orchestration layer and (v) IDE integration. The system is deployed on Amazon Web Services using a small set of Lambda functions fronted by Amazon API Gateway and scheduled by Amazon EventBridge. Vector storage and similarity search are provided by MongoDB Atlas Vector Search; sync state is held in Redis; large language model and embedding services are obtained from OpenAI. A VS Code and Cursor sidebar extension delivers the reconstructed intent into the developer's workflow.

The work is empirical and engineering-led: every component described in the report has been implemented, deployed and exercised against real Jira and GitHub data of the employing organisation. The contribution is therefore both methodological (an architecture for intent reconstruction) and operational (a working production-grade system).

1.5 Summary of Contributions

The dissertation makes the following contributions.

- It defines intent reconstruction as a distinct problem from traditional traceability link recovery, and frames it as a retrieval-and-synthesis task amenable to multi-query RAG.
- It proposes a five-layer cloud-hosted architecture in which a single ingestion Lambda handles two heterogeneous sources (Jira and GitHub) using shared embedding and vector-store machinery but independent state.
- It designs a multi-query retriever that fans the user's question and the local commit subjects out across two MongoDB Atlas vector indexes in parallel and merges the results using a max-pool re-ranking strategy.
- It introduces an agent orchestration layer that decomposes intent reconstruction into specialised agents (traceability, code-intent, reproduction, validation) coordinated by a router.
- It implements a VS Code and Cursor extension paired with a loopback-only local git agent so that the reconstructed intent is delivered with strong privacy and zero context-switching.
- It documents the deployment, configuration, security and operating costs of the resulting system in sufficient detail to be reproduced by other organisations.

1.6 Organisation of the Report

Chapter 2 surveys the relevant literature in requirement traceability, RAG-based traceability, transformer models for code and agentic AI for software engineering. Chapter 3 formalises the problem, the dissertation objectives and the scope. Chapter 4 develops the high-level system architecture and its five layers in detail. Chapter 5 describes the cloud-hosted implementation on AWS with annotated screenshots from

the deployed system. Chapter 6 explains the multi-source RAG and multi-agent reasoning pipeline. Chapter 7 presents the VS Code and Cursor chat extension and its end-to-end data flow. Chapter 8 discusses evaluation, observations and the validation strategy. Chapter 9 documents the engineering challenges and limitations encountered. Chapter 10 concludes the work and suggests future research directions. References, appendices, a glossary and the BITS submission checklist follow.

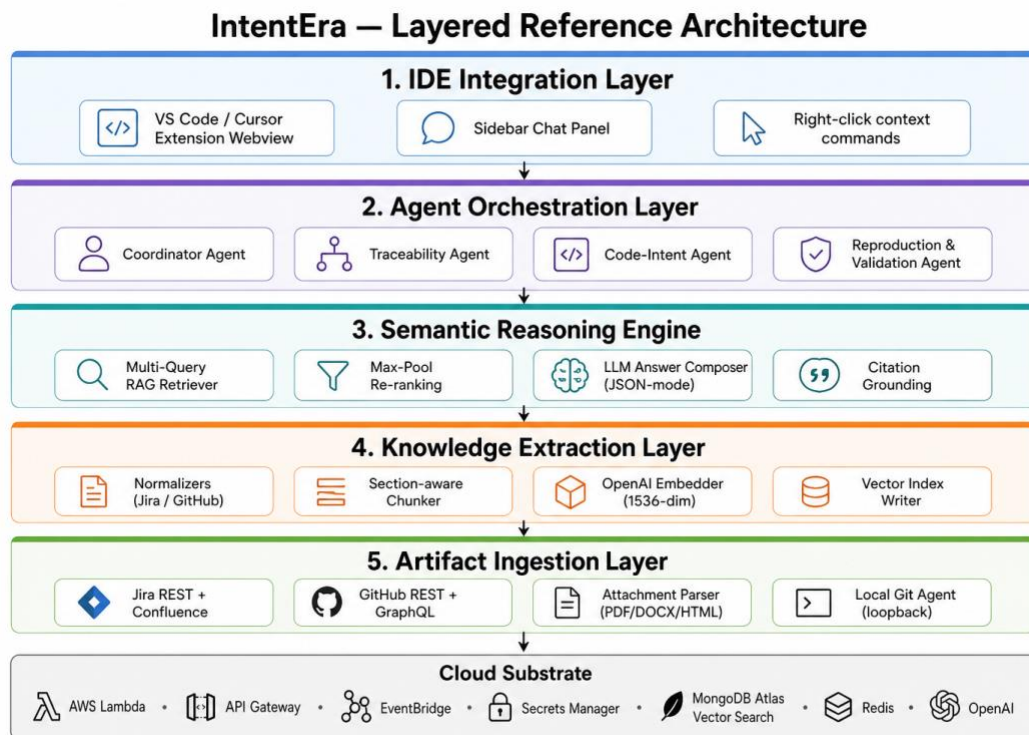


Figure 1: *IntentEra layered reference architecture, showing the five logical layers of the system and the underlying cloud substrate they share.*

Chapter 2 - Literature Survey

This chapter situates IntentEra in the existing body of work in requirement traceability, semantic code understanding, retrieval-augmented generation (RAG) and agentic AI for software engineering. The literature is broad; the survey here focuses on the threads that directly informed the architectural and algorithmic decisions taken in the dissertation.

2.1 Natural-Language Processing for Requirement Traceability

Guo, Steghöfer, Vogelsang and Cleland-Huang [1] survey the use of NLP techniques for requirement traceability, with a particular emphasis on how structured semantic entities can be extracted from informal natural-language requirements. Their central observation is that purely lexical approaches - such as keyword search or term-frequency similarity - routinely fail to capture the subtle phrasing differences between how a business analyst describes a requirement and how an engineer names a function or commits a change. They demonstrate that preprocessing pipelines that explicitly recognise entities (actors, actions, objects) materially improve downstream link quality. IntentEra adopts this insight at its preprocessing stage. The Jira normaliser converts Atlassian Document Format into clean text, separates structured fields (key, summary, status, labels) from prose, and emits one chunk per comment. This normalised representation is closer to the entity-aware representation advocated by Guo et al. [1] and provides a stronger basis for embedding-based retrieval than the raw exported JSON.

2.2 Empirical Comparison of Requirement-to-Code Link Recovery

Wang, Zou, Wan and others [2] provide a rigorous empirical study of state-of-the-art methods for requirement-to-code traceability link recovery, comparing classical information-retrieval (IR) techniques such as Vector Space Models and BM25 against deep-learning approaches based on word embeddings, sentence encoders and pretrained transformer models. Their headline finding is that no single technique dominates across all industrial datasets, and that hybrid approaches combining IR signals with neural similarity tend to outperform either family in isolation.

Two implications shaped IntentEra. First, the system retains structured metadata (ticket keys, file paths, commit SHAs, branch lists) alongside the embedding so that classical filters can be applied during \$vectorSearch - in effect, a hybrid IR + neural query. Second, by treating the user's question and each local commit subject as separate queries, the multi-query retriever is implicitly an ensemble, exploiting the empirical observation that no single signal is reliably best.

2.3 Retrieval-Augmented Generation for Traceability

Ali, Naganathan and Bork [3] explicitly apply Retrieval-Augmented Generation (RAG) to the problem of establishing traceability between natural-language requirements and software artifacts. Their architecture uses a sentence encoder to retrieve candidate links and then a large language model to compose a justification in natural language. This pattern - retrieve, then generate with retrieved context as grounding - forms the conceptual blueprint for the IntentEra reasoning engine.

RAG is preferred over end-to-end fine-tuning for three reasons that are particularly important in industrial deployments. First, labelled traceability data are scarce and

expensive to curate. Second, the underlying repositories evolve continuously, which makes statically fine-tuned weights stale. Third, RAG supports explicit citation and grounding, which is essential to mitigate the well-known hallucination risk of LLMs in technical contexts. IntentEra strengthens this further by forcing the LLM into JSON-mode and by post-filtering any cited identifier (commit SHA, Jira key, PR number) that does not appear in the supplied context.

2.4 Transformer Models for Code

Ahmad, Agha, Ray and others [4] survey the family of transformer models adapted to source code, including encoders such as CodeBERT and CodeT5 and decoder-style code completion models. Their survey establishes that transformers can capture structural regularities of code (identifier conventions, control flow patterns, type usage) and project them into a continuous vector space that is amenable to similarity search.

Kanade, Agrawal, de Melo and others [5] complement this by showing that self-training with unlabelled code substantially improves downstream tasks, suggesting that the embedding spaces produced by such models can be reused across many software engineering tasks without per-project fine-tuning.

IntentEra does not currently fine-tune a code-specific encoder. The first deployment uses OpenAI's text-embedding-3-small model (1536 dimensions) for both natural-language and code-adjacent artifacts (commit messages, PR bodies, issue bodies). This decision was deliberate: the artifacts indexed by IntentEra are predominantly natural-language explanations of code rather than raw source, and a high-quality general-purpose text encoder is more appropriate. The system is, however, structured

so that the embedder is a single pluggable component; replacing it with a code-specialised model would change a single class.

2.5 Agentic AI for Software Engineering

The recent surge of “agentic” AI systems - in which a coordinator decomposes a user goal into specialised sub-tasks executed by tool-using sub-agents - has produced a small but rapidly growing literature on agent-based developer tools. The recurring lesson from the existing systems is that decomposition by concern improves both interpretability and robustness: each agent can be evaluated and improved independently, and the coordinator can fall back gracefully if a specialist returns low-confidence output.

IntentEra adopts this principle at the orchestration layer. The design space includes a Traceability Agent (resolves requirement-to-code links), a Code-Intent Agent (composes the “why” explanation), a Reproduction Agent (derives repro steps from tickets and commits) and a Validation/Test Agent (suggests how to verify the fix). The current production deployment runs the first two agents end-to-end, with the remaining two implemented as scaffolds for the future-work phase.

2.6 Positioning and Gap

Reading the literature side by side, four observations stand out. (i) NLP-based traceability work focuses on link recovery rather than on explanation. (ii) RAG approaches tend to be evaluated on small public datasets rather than on production engineering corpora. (iii) Agentic systems for software engineering tend to focus on autonomous code generation rather than on understanding existing code. (iv) Almost no

work in the intersection has been packaged as a deployable, IDE-integrated system that an engineering team can adopt incrementally.

IntentEra contributes precisely at this intersection. It treats intent reconstruction (not link recovery) as the goal; it uses RAG over real production artifacts (Jira, Confluence, GitHub) and not synthetic datasets; it is structured as an agentic framework optimised for explanation rather than autonomous action; and it is deployed end-to-end on AWS with a first-class IDE integration. The remainder of this report describes that contribution.

Chapter 3 - Problem Statement, Objectives and Scope

3.1 Formal Problem Statement

Let R denote the set of requirement-bearing artifacts in an organisation - in the present work, Jira tickets, Confluence pages and GitHub issues. Let C denote the set of code-related artifacts - commits, pull requests, review threads. Let D denote the developer's working tree at a point in time, with a selected file f and (optionally) a line range $\langle a, b \rangle$. Given a natural-language question q from the developer about (f, a, b) , the problem is to construct an explanation E that answers q in plain language and is supported by a citation set $S \subseteq R \cup C$ such that every citation in S is grounded in a real artifact and every claim in E is attributable to citations in S .

Three properties make this formulation distinctive compared to classical traceability link recovery. First, the output is an explanation, not a binary link decision. Second, the input carries both a question and a precise code context (file, range), which is much richer than asking “which requirement maps to this code?” in the abstract. Third, the system is required to produce a grounded answer rather than the most-similar artifact, so retrieval and generation are not independent.

3.2 Dissertation Objectives

The objectives crystallise from the formal statement above.

1. To investigate and characterise the phenomenon of intent erosion in production-grade codebases, with concrete examples drawn from the employing organisation.
2. To design a semantic reconstruction framework that establishes meaningful relationships between natural-language requirements and code changes, supporting both metadata-based and embedding-based linkage.

3. To architect a cloud-hosted agentic AI system that combines Retrieval-Augmented Generation with multi-agent reasoning to infer, synthesise and explain implementation intent.
4. To integrate the system with modern Integrated Development Environments (VS Code and Cursor) so that intent reconstruction is delivered without context-switching.
5. To enable automated traceability workflows: mapping requirements to commits, identifying relevant code segments and generating contextual explanations of how and why specific changes address requirements or defects.
6. To support issue reproduction and validation reasoning, allowing developers to understand the conditions under which a problem occurs and to verify whether a given implementation resolves it.
7. To evaluate the deployed system on functional correctness, operating cost and qualitative impact on developer cognitive load.

3.3 Scope of Work

The scope of the dissertation is bounded as follows. The system ingests Jira (cloud), Confluence (cloud) and GitHub repositories. It does not currently ingest other ITSM platforms (such as ServiceNow), other source control hosts (such as Bitbucket Cloud or Server beyond proof-of-concept), or chat platforms (such as Slack). The intent reconstruction is performed for the two source families demonstrated in production: Jira and GitHub. The IDE integration ships as a single VSIX that installs in both VS Code and Cursor.

Out of scope are: code generation or autonomous repair; fine-tuning of bespoke LLMs on the organisation's data; full static-analysis-based call-graph reasoning; on-premises deployment without internet egress (the design assumes managed cloud services). These are sketched in the future-work chapter as natural extensions.

3.4 Success Criteria

The dissertation considers itself successful if it meets:

- Functional: the deployed system answers a representative developer question by retrieving grounded artifacts from both Jira and GitHub stores and synthesising a citation-backed explanation in less than the configured request timeout.
- Operational: the system runs continuously on an AWS account with predictable cost, automated incremental sync and no manual intervention on the daily incremental cycle.
- Architectural: the system decomposes cleanly into the five layers proposed in Chapter 4, and a new source can be added by implementing only a fetcher plus a normaliser.
- Methodological: the framework is reproducible from this report and the accompanying repository documentation, including step-by-step deployment instructions.

3.5 Plan of Work

Table 1: *Phased plan of work for the dissertation, including completion status as of the final submission.*

Phase	Dates	Deliverable	Status
Outline & Proposal	20 Jan 2026 - 07	Literature survey, dissertation	Completed

Finalisation	Feb 2026	outline, problem formulation.	
Requirement Analysis & Architecture	08 Feb 2026 - 28 Feb 2026	Cloud-hosted agentic framework design; identification of integrations; semantic reconstruction pipeline definition.	Completed
Prototype Phase I - Artifact Integration	01 Mar 2026 - 20 Mar 2026	Implementation of Jira and GitHub fetchers; commit/ticket/PR extraction; metadata traceability.	Completed
Midterm Report & Submission	21 Mar 2026 - 28 Mar 2026	Documentation of progress, architecture and prototype status; midterm dissertation report.	Completed
Prototype Phase II - Agentic AI Reasoning	29 Mar 2026 - 20 Apr 2026	Multi-query retriever, multi-agent orchestration, JSON-mode LLM answer composition with citations.	Completed
Testing, Validation & User Evaluation	21 Apr 2026 - 05 May 2026	Functional testing on production repositories; cognitive-load and traceability accuracy evaluation; chat extension on real files.	Completed
Final Dissertation	06 May 2026 - 11 May 2026	Consolidation of findings,	Completed

Writing & Review	May 2026	supervisor feedback, final manuscript.	
Final Submission	12 May 2026	Final review and official submission of the dissertation report.	Completed

Chapter 4 - System Architecture

4.1 Architectural Principles

The architecture of IntentEra is shaped by five principles that were derived from the literature survey and from the operational constraints of the employing organisation.

- Layered separation of concerns: ingestion, extraction, reasoning, orchestration and IDE delivery are independently implementable, testable and replaceable.
- Source neutrality: each artifact source (Jira, GitHub) plugs into the same downstream embedding and vector-store pipeline; adding a new source touches only a fetcher and a normaliser.
- Idempotence: every chunk written to the vector store has a deterministic identifier, so re-running the same ingestion is a safe, low-cost operation.
- Strong grounding: the LLM never speaks without a context package; citations not present in the context are dropped before the response is returned to the developer.
- Cloud-native economics: the system pays per-invocation rather than per-server, which makes it viable for teams with bursty developer-question workloads.

4.2 The Five Layers

Figure 1 summarises the five layers. We elaborate each below.

4.2.1 Artifact Ingestion Layer

The ingestion layer establishes secure, structured access to external systems through APIs and connectors. For Jira it uses Basic authentication with email plus API token over the public REST API, optionally following Confluence “remote link” URLs to fetch additional context. For GitHub it uses a Personal Access Token with repository scope, combining REST and GraphQL endpoints to obtain commits on active branches, pull

requests with review threads, and issues with comments. The layer respects rate limits, retries with exponential back-off and emits structured logs that downstream observability tools can reason about.

4.2.2 Knowledge Extraction Layer

Raw artifacts are converted into machine-processable semantic representations in this layer. Atlassian Document Format is rendered to clean text. Attachments in PDF, DOCX, TXT and HTML are parsed and extracted. Pull-request reviews are grouped with their inline comments so that reviewer rationale stays co-located with the code lines it discusses. The output is passed through a section-aware chunker (Figure 8) that produces “one idea per chunk” - each chunk is long enough to be self-contained and short enough to be embedded efficiently. Each chunk is then embedded with OpenAI text-embedding-3-small (1536 dimensions) and written into the appropriate MongoDB Atlas vector collection.

4.2.3 Semantic Reasoning Engine

The reasoning engine is the core intelligence of the system. It performs cross-artifact correlation: given a developer question and a set of local commit subjects, it embeds them in a single OpenAI batch, fans the resulting query vectors out across the Jira and GitHub vector indexes in parallel, max-pools the results by chunk identifier and asks an LLM in JSON-mode to compose a non-technical “why” explanation grounded in the merged context. Citations not present in the context are stripped before the response is returned. The detail of this pipeline is the subject of Chapter 6.

4.2.4 Agent Orchestration Layer

Agentic decomposition turns a single monolithic prompt into a collaboration between specialised reasoners (Figure 5). A Coordinator Agent receives the developer's request, decides which specialists to consult and synthesises their outputs into a single response. The current production design implements the Coordinator and the Code-Intent Agent end-to-end, with the Traceability, Reproduction and Validation/Test agents wired up as scaffolds whose internal prompts are being matured. The decomposition makes prompts shorter and more specific, which in turn improves both response quality and observability.

4.2.5 IDE Integration Layer

The IDE layer presents the reconstructed intent to the developer without forcing them to leave their editor. The implementation ships as a single VSIX (the same artifact installs into both VS Code and Cursor) backed by a sidebar webview, two right-click commands (“IntentEra: Ask about selection” and “IntentEra: Ask about this file”) and a small loopback Local Git Agent that extracts commit history from the user’s working tree on demand. Chapter 7 covers the extension in detail.

4.3 Cross-Cutting Services

Three cross-cutting services support every layer. AWS Secrets Manager holds a single pooled JSON secret containing every credential the Lambdas need (Jira, Confluence, GitHub, MongoDB, Redis, OpenAI). Redis stores per-source synchronisation state and a short-lived distributed lock, so two overlapping ingestion invocations cannot trample each other. CloudWatch captures structured JSON logs from every Lambda; the three IntentEra log groups are visible in Figure 9.

4.4 Significance of the Design

The architecture shifts software engineering support from a document-centric retrieval model (“which page should I read?”) to an intent-centric reasoning model (“why is this code the way it is?”). The five-layer decomposition is significant for three reasons: (i) it makes the system extensible - new sources, new agents and new IDEs can be added independently; (ii) it makes the system testable - each layer has a small, well-typed interface; and (iii) it makes the system economical - idempotent incremental ingestion and pay-per-invocation Lambdas keep the monthly cost in single-digit US dollars for the workloads exercised so far.

Chapter 5 - Cloud-Hosted Implementation on AWS

This chapter walks through the production deployment of IntentEra on AWS. The objective is to demonstrate that the abstract architecture of Chapter 4 has a concrete, operational realisation, and to make the deployment reproducible.

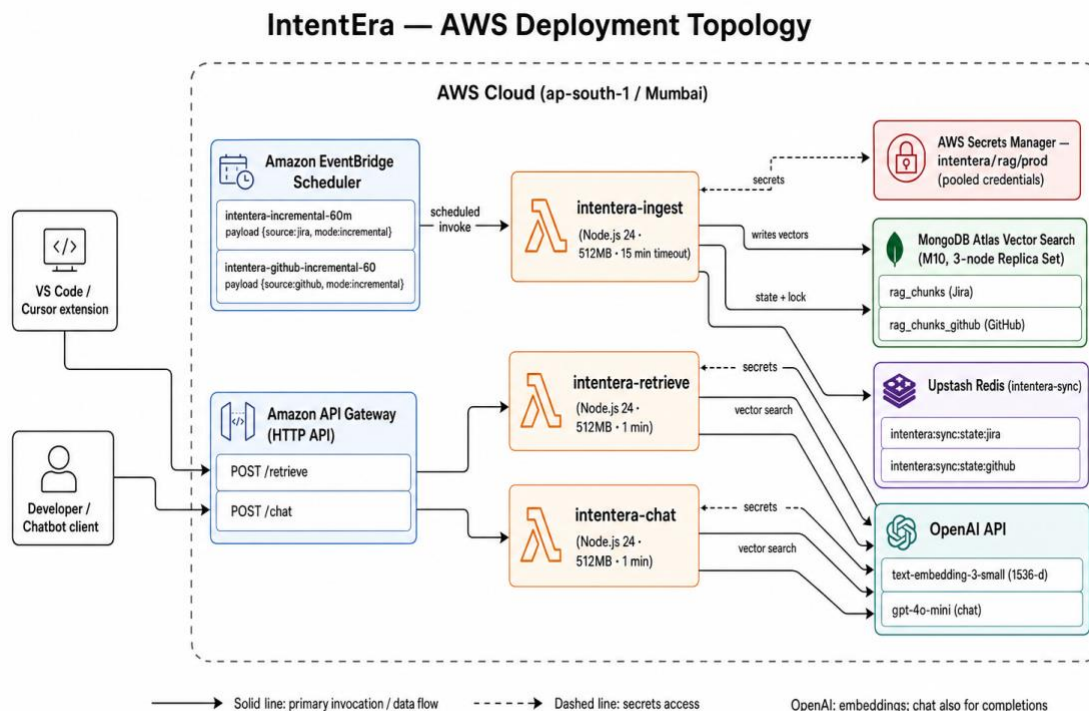


Figure 2: AWS deployment topology: EventBridge schedules the ingestion Lambda; API Gateway routes /retrieve and /chat to dedicated Lambdas; all Lambdas pull pooled credentials from Secrets Manager and share MongoDB Atlas, Redis and OpenAI.

5.1 Compute: Three AWS Lambda Functions

The system runs on three Node.js 24 Lambda functions, each deployed from the same code artifact but invoking a different handler entry point. Figure 2 in this chapter shows the console listing.

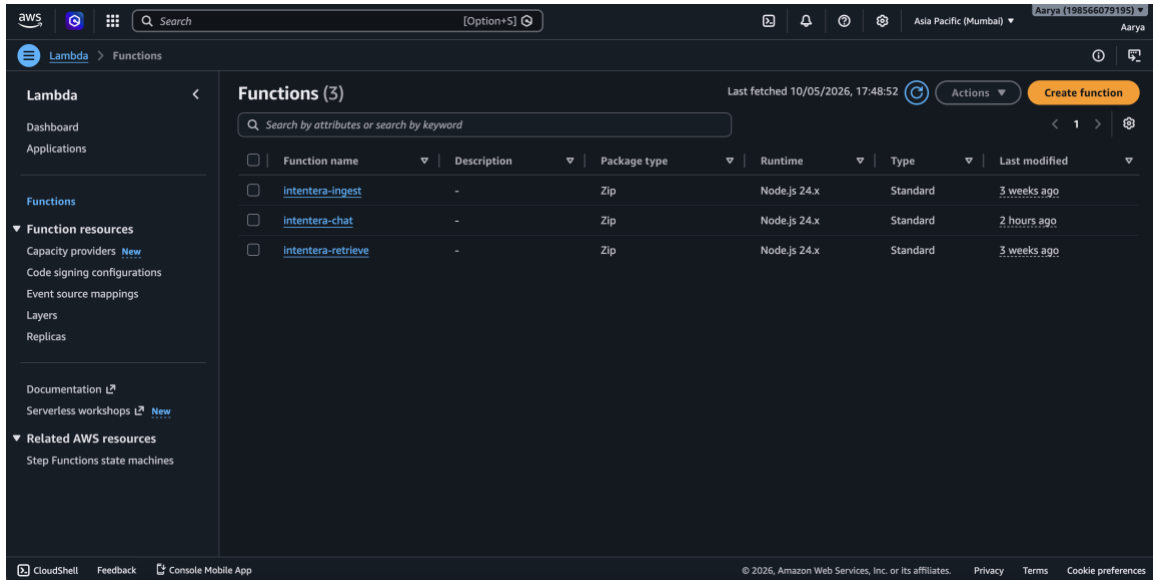


Figure 3: AWS Lambda console listing the three IntentEra functions: *intentera-ingest* (scheduled ingestion), *intentera-retrieve* (retrieval API) and *intentera-chat* (chat API).

Each function is sized for its purpose. The ingestion Lambda (Figure 3) is configured with a 15-minute timeout because a full import can pull thousands of artifacts; the chat Lambda (Figure 4) is configured with a one-minute timeout because a single request is bounded by the LLM call; the retrieve Lambda is similarly short-lived. The current memory allocation is 512 MB; this was chosen empirically and provides ample headroom for the embedding-batch and vector-search workloads.

intentera-ingest

Throttle Copy ARN Actions

Function overview info

Diagram Template

intentera-ingest

Layers (0)

+ Add trigger

+ Add destination

Export to Infrastructure Composer Download

Function ARN
arn:aws:lambda:ap-south-1:198566079195:function:intentera-ingest

Description
-

Last modified
3 weeks ago

Code Test Monitor Configuration Aliases Versions

Configuration

General configuration info

Description
-

Memory
512 MB

Ephemeral storage
512 MB

Timeout
15 min 0 sec

SnapStart
None

Edit

General configuration
Triggers
Permissions
Destinations
Function URL
Environment variables
Tags
VPC
RDS databases
Monitoring and operations tools
Concurrency and recursion detection
Asynchronous invocation
Code signing
File systems
State machines

Figure 4: *intentera-ingest* general configuration: 512 MB memory, 15-minute timeout, 512 MB ephemeral storage.

intentera-chat

Throttle Copy ARN Actions

Function overview info

Diagram Template

intentera-chat

Layers (0)

API Gateway

+ Add trigger

+ Add destination

Export to Infrastructure Composer Download

Function ARN
arn:aws:lambda:ap-south-1:198566079195:function:intentera-chat

Description
-

Last modified
2 hours ago

Code Test Monitor Configuration Aliases Versions

Configuration

General configuration info

Description
-

Memory
512 MB

Ephemeral storage
512 MB

Timeout
1 min 0 sec

SnapStart
None

Edit

General configuration
Triggers
Permissions
Destinations
Function URL
Environment variables
Tags
VPC
RDS databases
Monitoring and operations tools
Concurrency and recursion detection
Asynchronous invocation
Code signing
File systems
State machines

Figure 5: *intentera-chat* general configuration: 512 MB memory, 1-minute timeout.

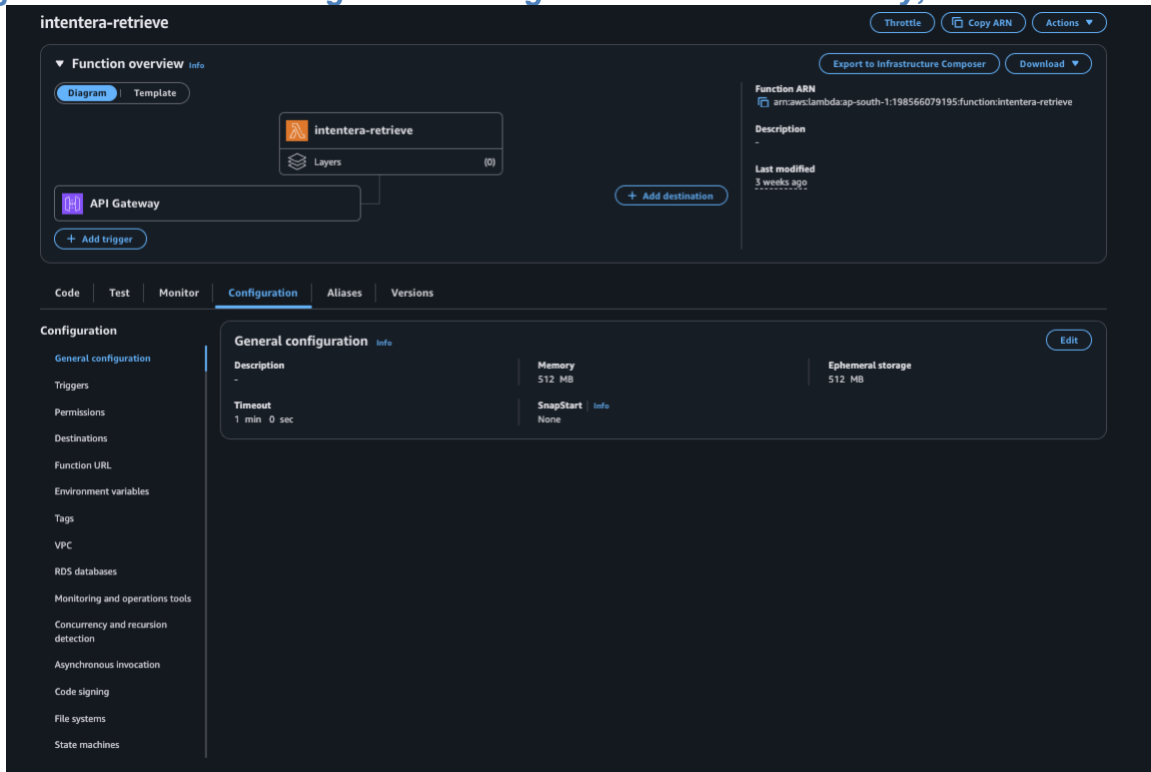


Figure 6: *intentera-retrieve* general configuration with the API Gateway trigger attached to the function.

5.2 Lambda Environment Variables

Each Lambda is configured with a small set of environment variables. Sensitive credentials are not stored as Lambda environment variables; instead, `USE_SECRETS_MANAGER=true` tells the function to read its credentials from a single pooled secret (see Section 5.4). This is visible in the console screenshots below.

intentera-ingest Throttle Copy ARN Actions

Function overview info

Diagram Template

 **intentera-ingest**

 Layers (0)

+ Add trigger + Add destination

Function ARN
arn:aws:lambda:ap-south-1:198566079195:function:intentera-ingest

Description
-

Last modified
3 weeks ago

Export to Infrastructure Composer Download

Configuration Aliases Versions

Configuration

- General configuration
- Triggers
- Permissions
- Destinations
- Function URL
- Environment variables**
- Tags
- VPC
- RDS databases
- Monitoring and operations tools
- Concurrency and recursion detection
- Asynchronous invocation
- Code signing
- File systems
- State machines

Environment variables (5) Edit

The environment variables below are encrypted at rest with the default Lambda service key.

Key	Value
INCREMENTAL_LOOKBACK_MINUTES	60
LOG_LEVEL	info
SECRETS_MANAGER_SECRET_ID	arn:aws:secretsmanager:ap-south-1:198566079195:secret:intentera/rag/prod-5CjBTF
SYNC_MODE	
SYNC_SOURCE	
USE_SECRETS_MANAGER	true

Figure 7: Environment variables for the intentera-ingest Lambda. Note USE_SECRETS_MANAGER=true and the Secrets Manager ARN; the actual credentials live in Secrets Manager rather than alongside the function configuration.

intentera-chat Throttle Copy ARN Actions

Function overview info

Diagram Template

 **intentera-chat**

 Layers (0)

 API Gateway

+ Add trigger + Add destination

Function ARN
arn:aws:lambda:ap-south-1:198566079195:function:intentera-chat

Description
-

Last modified
2 hours ago

Export to Infrastructure Composer Download

Configuration Aliases Versions

Configuration

- General configuration
- Triggers
- Permissions
- Destinations
- Function URL
- Environment variables**
- Tags
- VPC
- RDS databases
- Monitoring and operations tools
- Concurrency and recursion detection
- Asynchronous invocation
- Code signing
- File systems
- State machines

Environment variables (3) Edit

The environment variables below are encrypted at rest with the default Lambda service key.

Key	Value
LOG_LEVEL	debug
SECRETS_MANAGER_SECRET_ID	intentera/rag/prod
USE_SECRETS_MANAGER	true

Figure 8: Environment variables for the *intentera-chat* Lambda. The configuration is intentionally minimal so that secrets stay out of the function metadata.

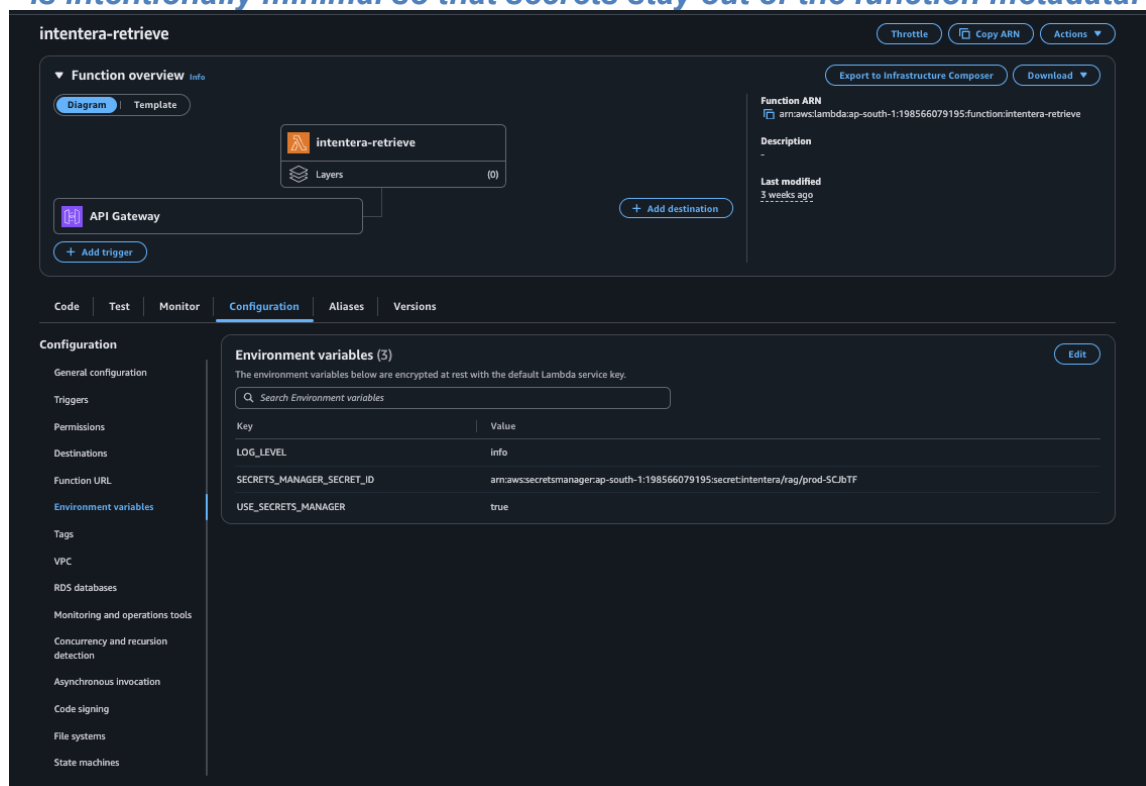


Figure 9: Environment variables for the *intentera-retrieve* Lambda.

5.3 Scheduling: Amazon EventBridge

Two EventBridge schedules drive the ingestion Lambda - one per source. Each schedule fires every 60 minutes and passes a small JSON payload that selects the source pipeline.

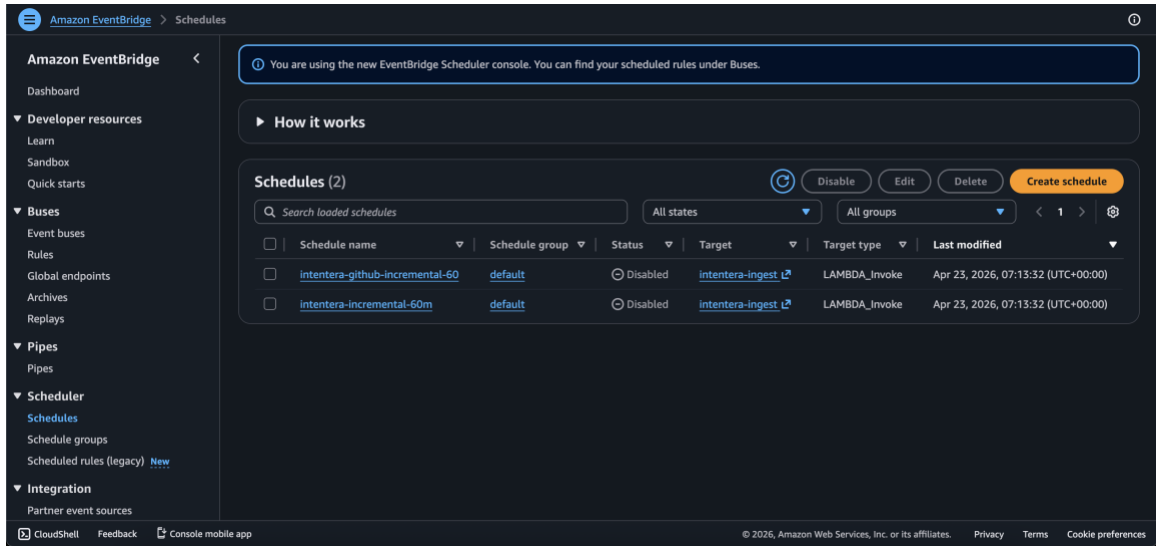


Figure 10: Two EventBridge schedules registered against the ingestion Lambda: *intentera-incremental-60m* for Jira and *intentera-github-incremental-60* for GitHub.

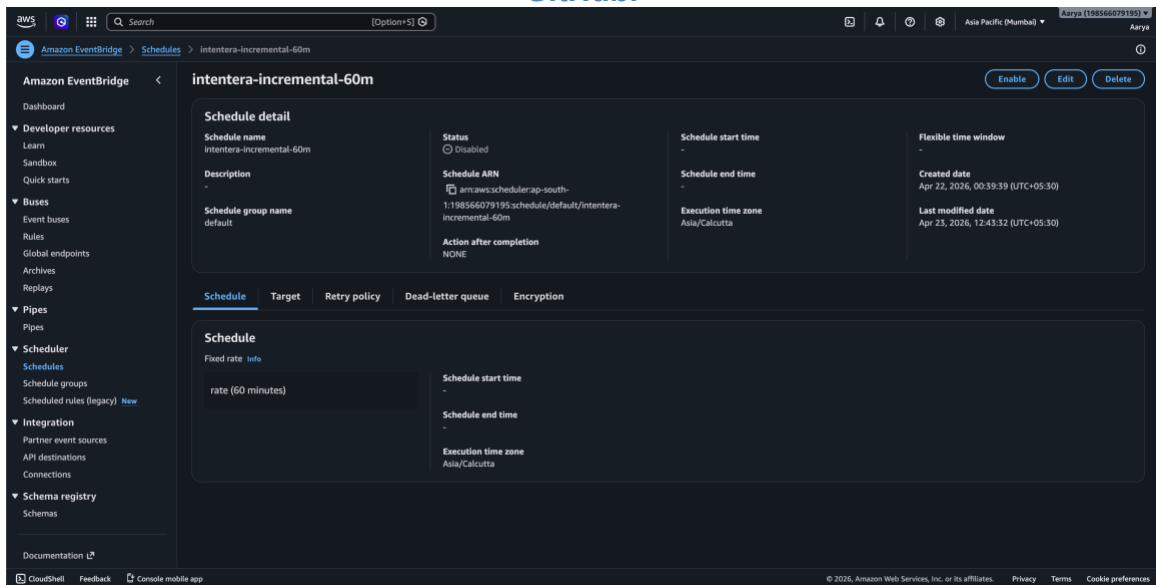


Figure 11: Detail of the Jira schedule (*intentera-incremental-60m*): a fixed-rate 60-minute trigger.

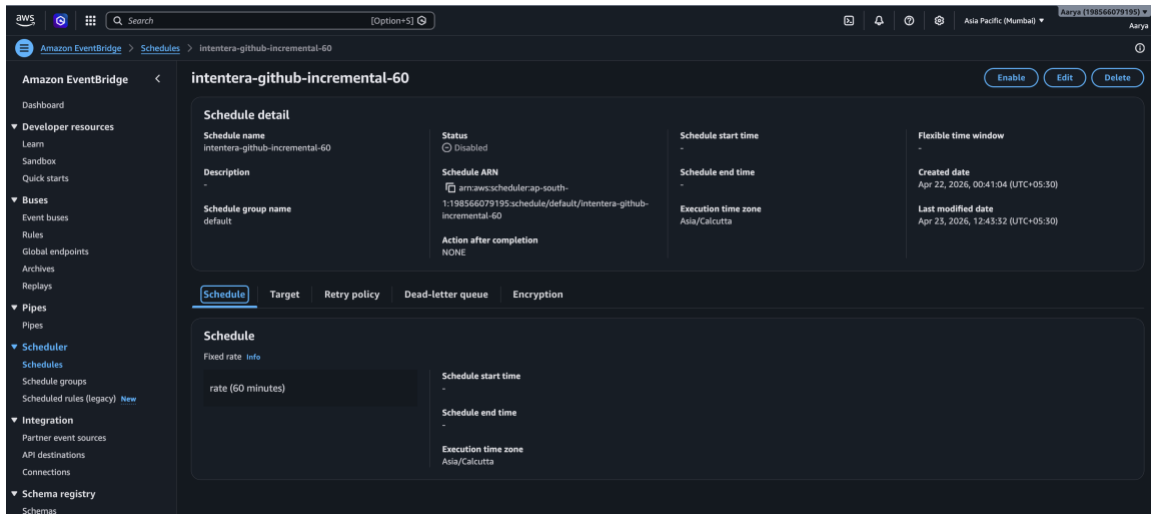


Figure 12: Detail of the GitHub schedule (*intentera-github-incremental-60*): the same fixed-rate cadence as the Jira schedule, but a different payload.

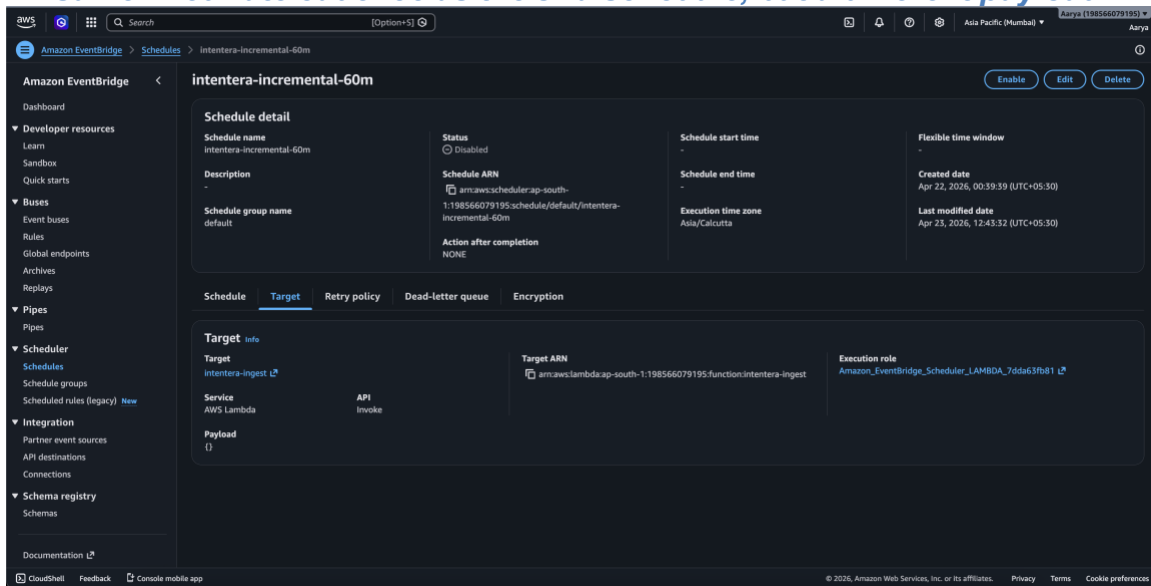


Figure 13: Target tab of the Jira schedule: AWS Lambda invocation of the *intentera-ingest* function.

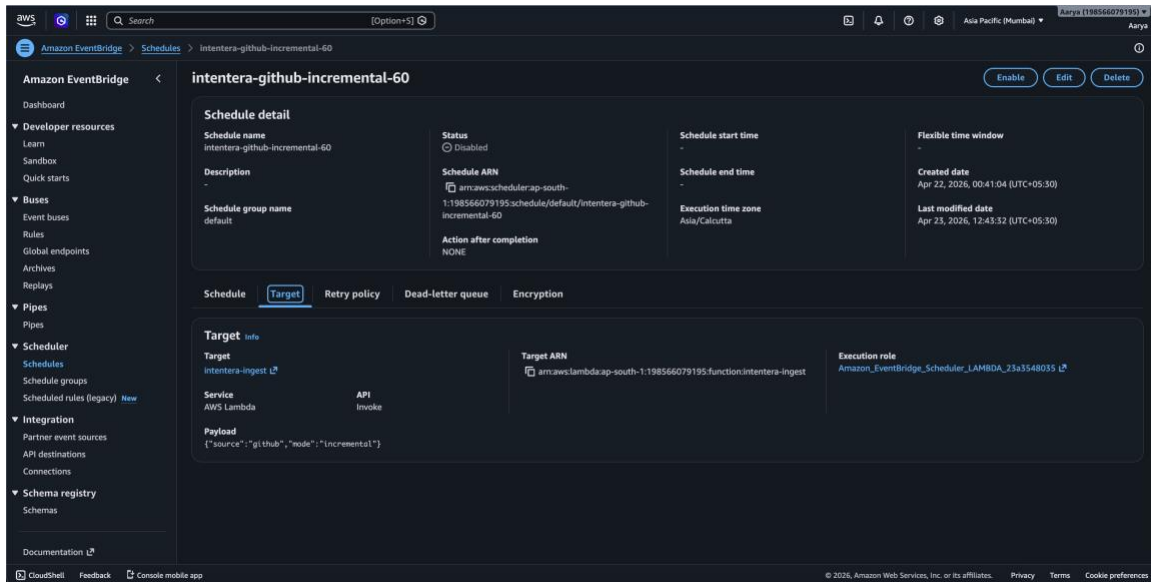


Figure 14: Target tab of the GitHub schedule: the same Lambda is invoked, but the payload `{"source": "github", "mode": "incremental"}` selects the GitHub pipeline at runtime.

Decoupling the schedules at the EventBridge layer (rather than scheduling a single Lambda that branches internally) is deliberate: it lets operators enable, disable, accelerate or decelerate either source independently, without code changes.

5.4 Credentials: AWS Secrets Manager

All credentials are consolidated in a single Secrets Manager secret, `intentera/rag/prod`, and pulled in on Lambda cold start. This single-secret approach was chosen over per-credential secrets because the Lambdas need most of the keys together, and the cost model of Secrets Manager favours consolidating related values.

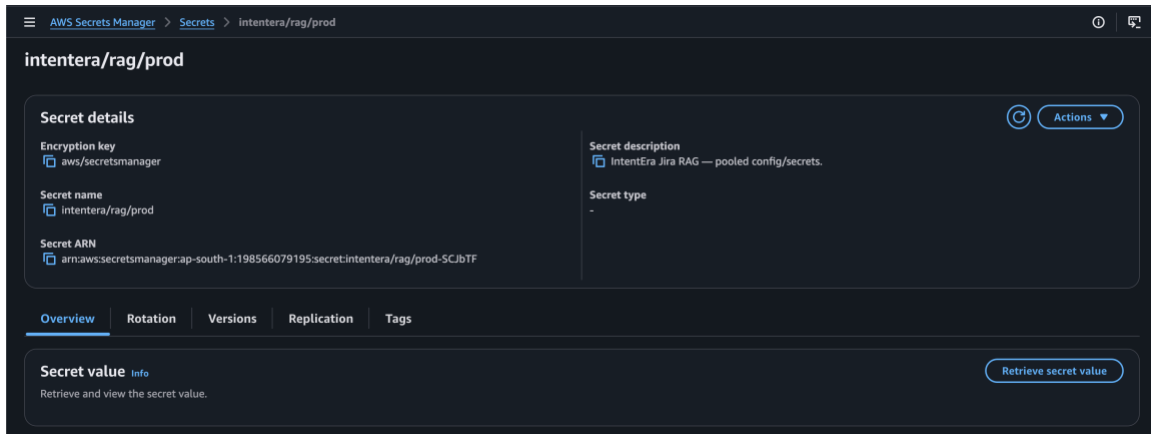


Figure 15: AWS Secrets Manager: the *intentera/rag/prod* secret holding the pooled JSON credential blob (Jira, Confluence, GitHub, MongoDB, Redis and OpenAI tokens).

Each Lambda's execution role is granted only `secretsmanager:GetSecretValue` on this single secret ARN, which tightens the blast radius if an execution role is ever compromised.

5.5 Front Door: Amazon API Gateway

Two HTTP routes are exposed: `POST /retrieve` and `POST /chat`. The former returns ranked hits and a ready-to-paste context string for any caller building their own LLM stack; the latter is the fully-formed chat endpoint consumed by the IDE extension.

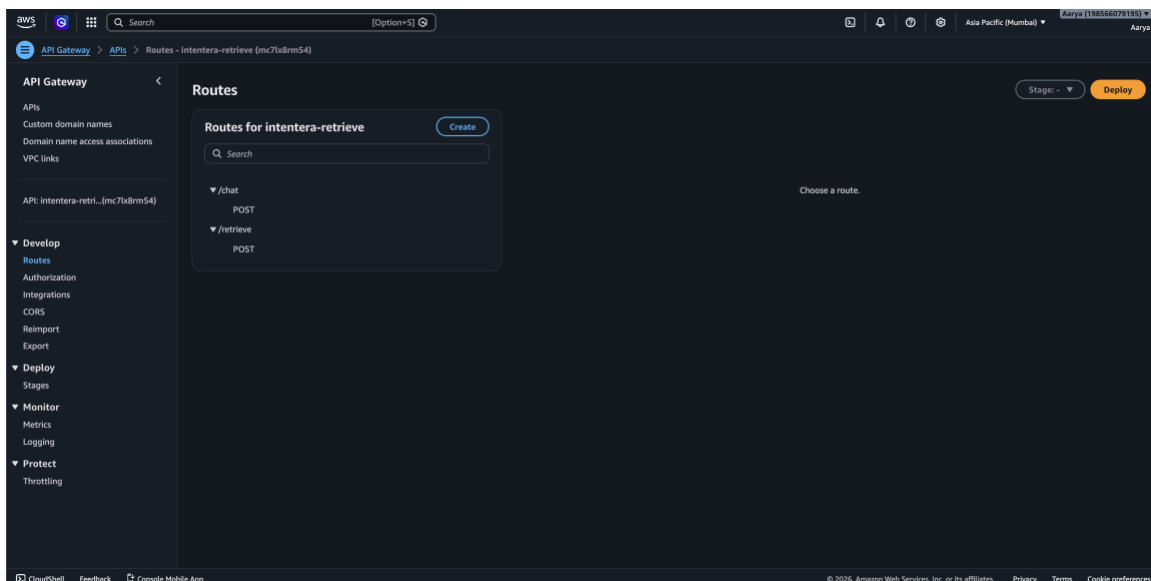


Figure 16: Amazon API Gateway routes: *POST /retrieve* and *POST /chat* both live on the same HTTP API, mapped to the corresponding Lambdas.

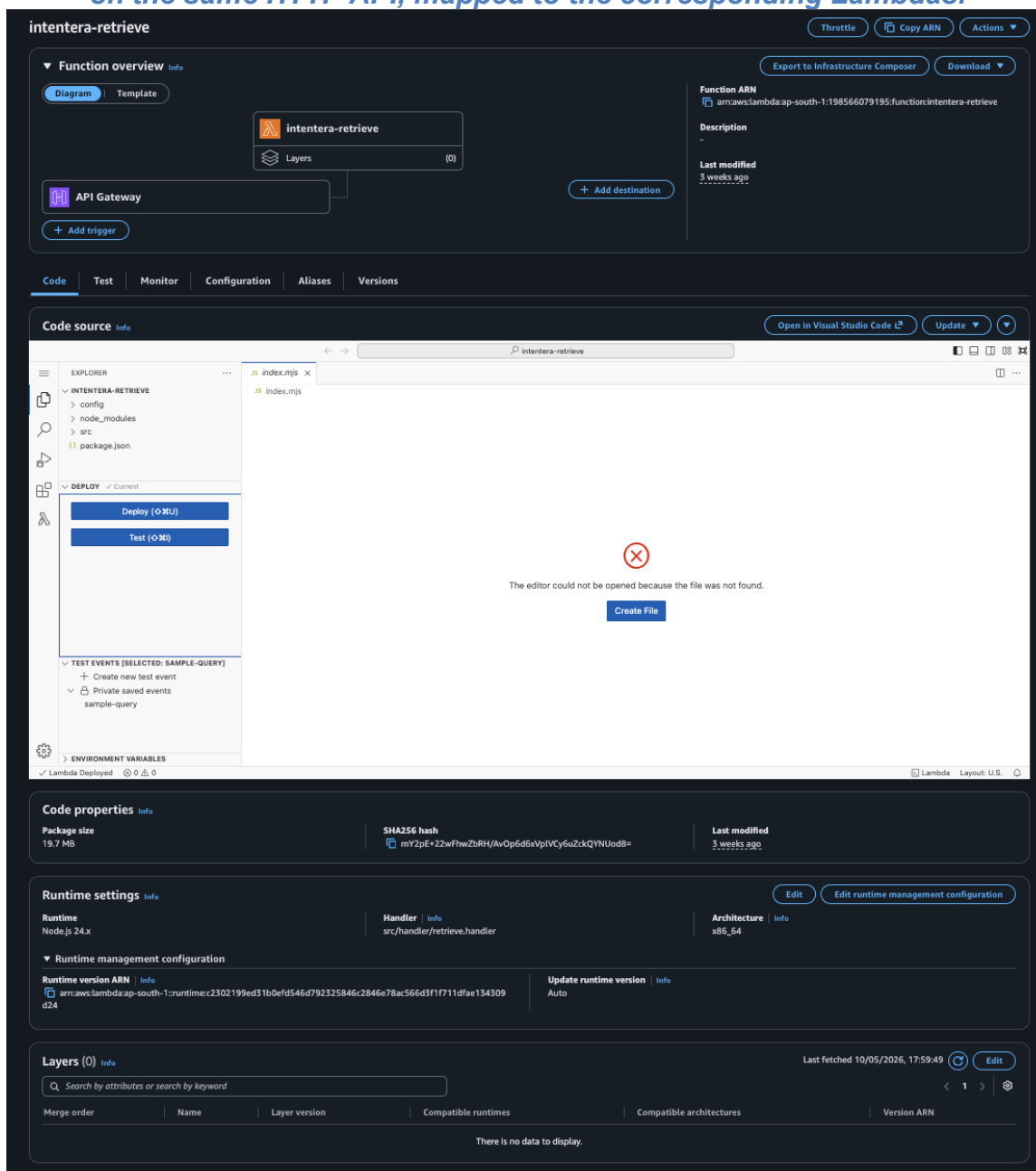


Figure 17: Code source view of the *intentera-retrieve* Lambda inside the AWS console editor.

Function overview

Diagram

Template

intentera-ingest

Layers (0)

+ Add trigger

+ Add destination

Function ARN

arn:aws:lambda:ap-south-1:198566079195:function:intentera-ingest

Description

-

Last modified

3 weeks ago

Export to Infrastructure Composer

Download

Code

Test

Monitor

Configuration

Aliases

Versions

Code source

Open in Visual Studio Code

Update

EXPLORER

INTENTERA-INGEST

config

node_modules

src

package.json

DEPLOY

Current

Deploy (OBU)

Test (OMI)

TEST EVENTS [SELECTED: FORCED-FULL]

Create new test event

Private saved events

forced-full

ENVIRONMENT VARIABLES

Lambda Deployed

0 0

Amazon Q

Meet Amazon Q

Get inline code suggestions

Amazon Q suggests code as you type based on your open files. Accepted suggestions from licensed code will go into the Code Reference Log.

Try Example

TIP: Invoke manually with `opt+alt+c`

Ask using chat

Check for security vulnerabilities

Access actions and options

Mark Done

```

1 // function to return all of the object keys in an S3 bucket
2 export const getKeys = async (bucketName, s3Client) => {
3   const params = {
4     Bucket: bucketName
5   };
6   const keys = [];
7   let isTruncated = true;
8   let continuationToken;
9   while (isTruncated) {
10    const response = await s3Client.listObjectsV2(params);
11    isTruncated = response.isTruncated;
12    params.continuationToken = response.continuationToken;
13    response.contents.forEach(item => keys.push(item.key));
14  }
15  return keys;
16 }
17

```

Code properties

Package size

19.7 MB

SHA256 hash

mY2pE+22wFhwZbRH/AwOp6d6XvplVCy6uZckQYNuod8=

Last modified

3 weeks ago

Runtime settings

Runtime

Node.js 24.x

Handler

src/handler/ingest.handler

Architecture

x86_64

Runtime management configuration

Runtime version ARN

arn:aws:lambda:ap-south-1::runtime:c2302199ed31b0efd546d792325846c2846e78ac566d3f1f711dfe134309d24

Update runtime version

Auto

Layers (0)

Search by attributes or search by keyword

1

Edit

Merge order

Name

Layer version

Compatible runtimes

Compatible architectures

Version ARN

There is no data to display.

Figure 18: Code source view of the *intentera-ingest* Lambda; package size, SHA256 hash and runtime settings are visible at the bottom.

31

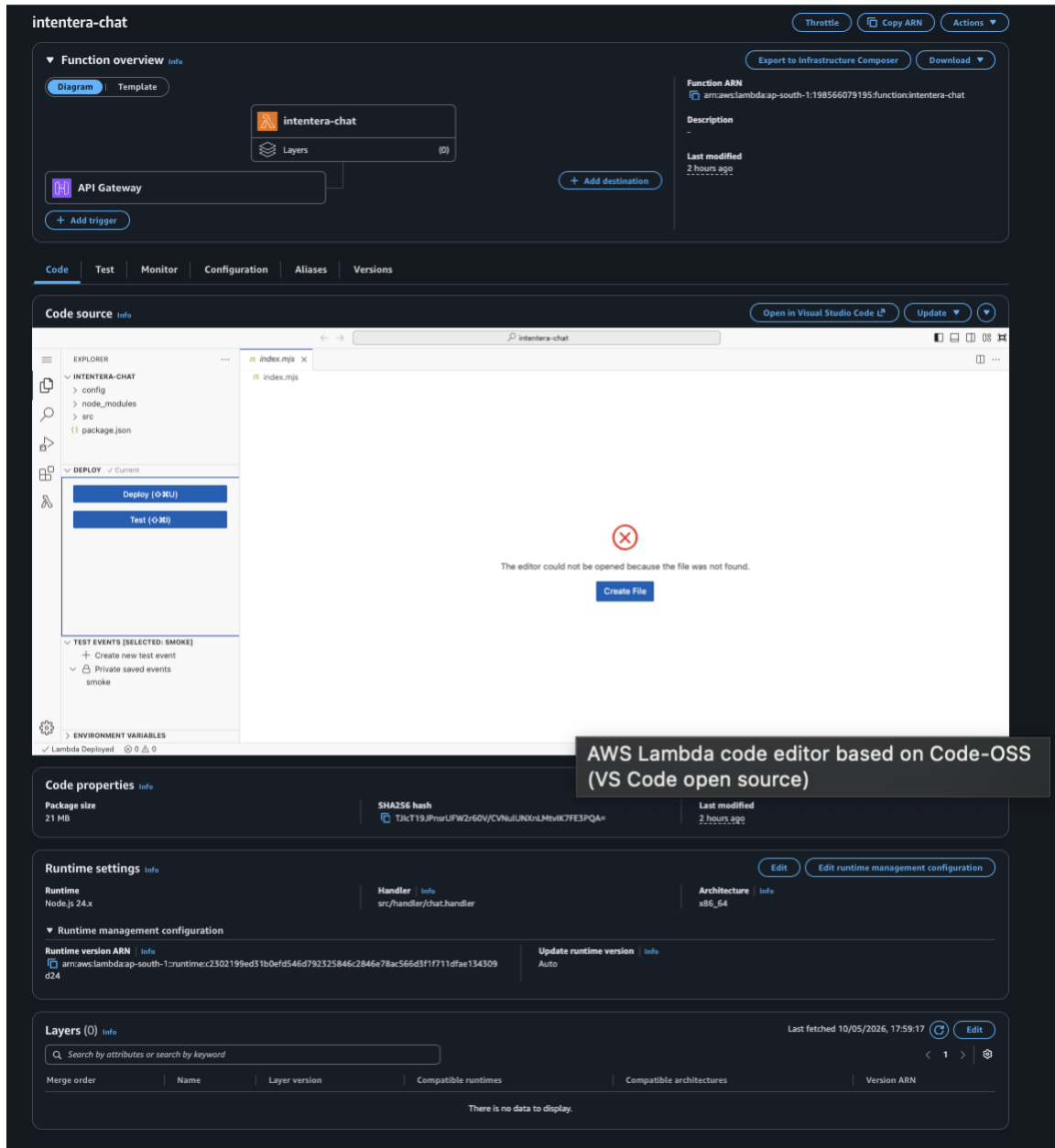


Figure 19: Code source view of the intentera-chat Lambda; deployment metadata and the package hash are visible at the bottom.

5.6 Vector Store: MongoDB Atlas Vector Search

MongoDB Atlas provides the vector index used by all \$vectorSearch queries. A single M10 cluster (three-node Replica Set in ap-south-2 / Hyderabad) hosts the IntentEra database with two collections: rag_chunks for Jira and rag_chunks_github for GitHub. Each collection has its own vector index (vector_index and vector_index_github) over

the 1536-dimensional embedding field, plus filter fields specific to its source (ticketKey, projectKey for Jira; entityType, repoFullName, branches, prNumber for GitHub).

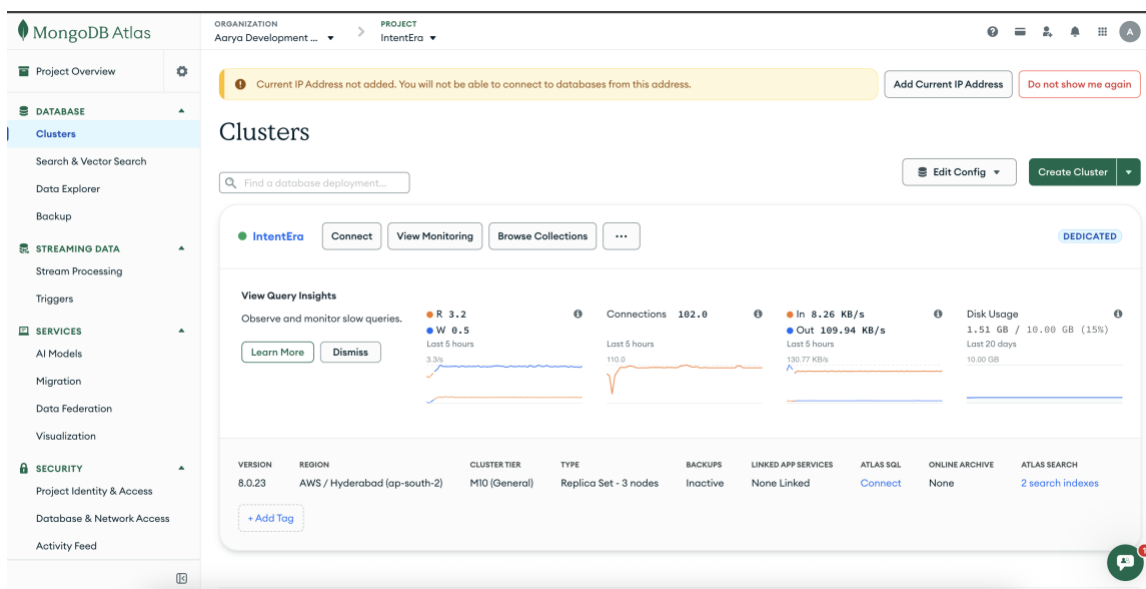


Figure 20: MongoDB Atlas: IntentEra cluster (M10, three-node Replica Set, Hyderabad region). The five-hour sparkline shows steady-state read/write traffic from the Lambdas.

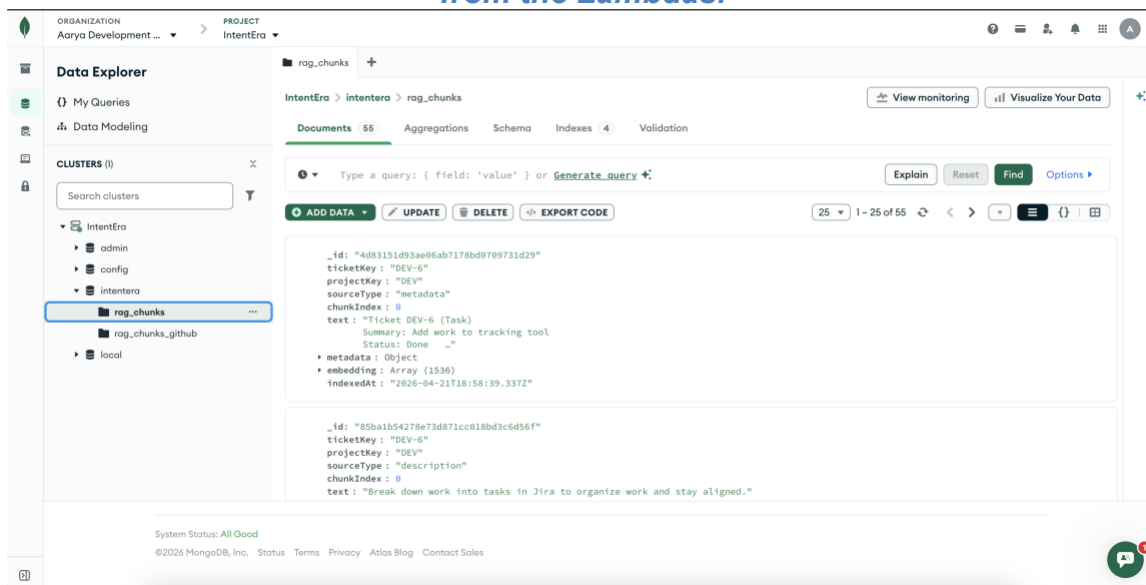


Figure 21: Sample documents in rag_chunks (Jira). Each document carries a deterministic _id, the ticketKey, the source type (metadata / description / comment / attachment), the chunk index, the chunk text and a 1536-element embedding array.

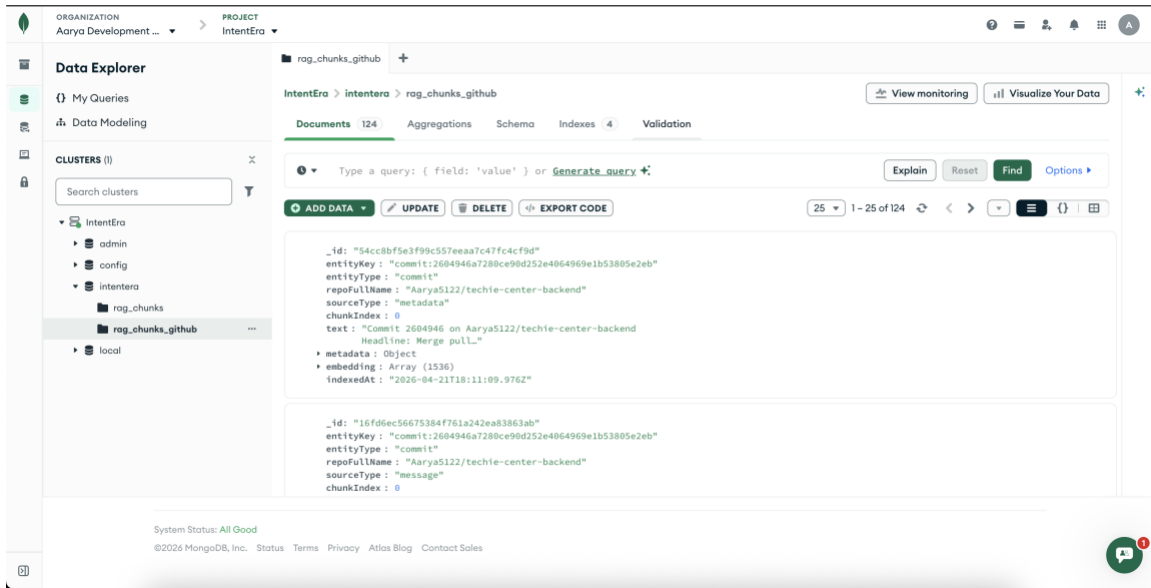


Figure 22: Sample documents in `rag_chunks_github` (GitHub). Each document carries the `entityKey` (e.g. `commit:<sha>`), the `entityType`, the `repo` full name, the `chunk` slot and the `embedding` array.

5.7 Sync State: Upstash Redis

Redis stores two tiny JSON blobs - one per source - that record the last successful run timestamps, the set of known entity identifiers and the run status. Together with the short-lived `intentera:sync:lock:{source}` key, this is sufficient to make incremental ingestion safe under overlapping invocations.

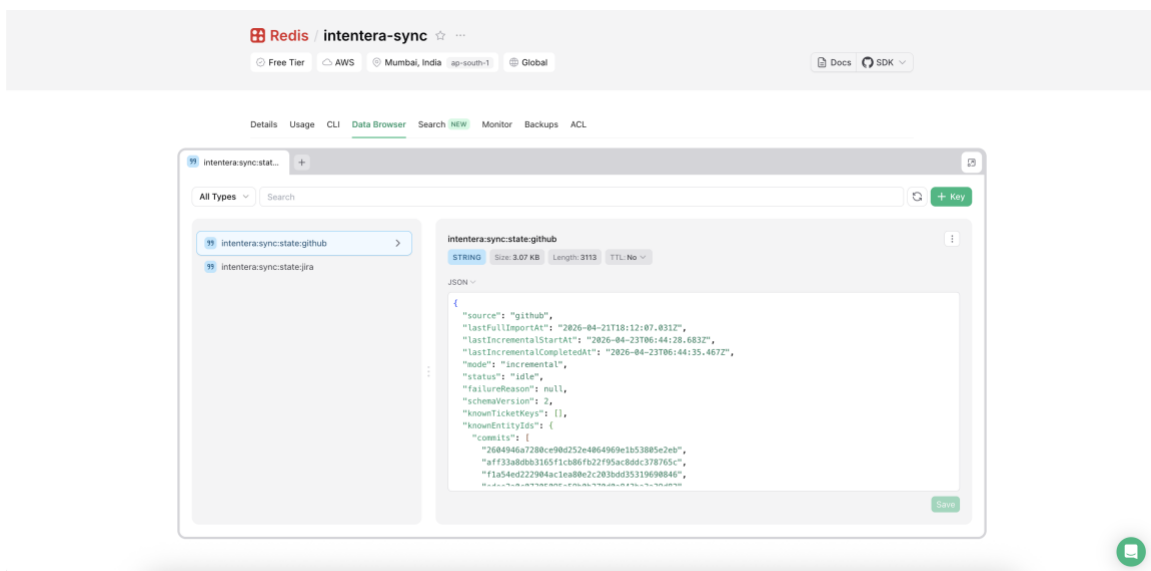


Figure 23: Upstash Redis (intentera-sync) data browser showing the `intentera:sync:state:github` key. The JSON value records the last full and incremental timestamps, the run status and the set of known commit identifiers.

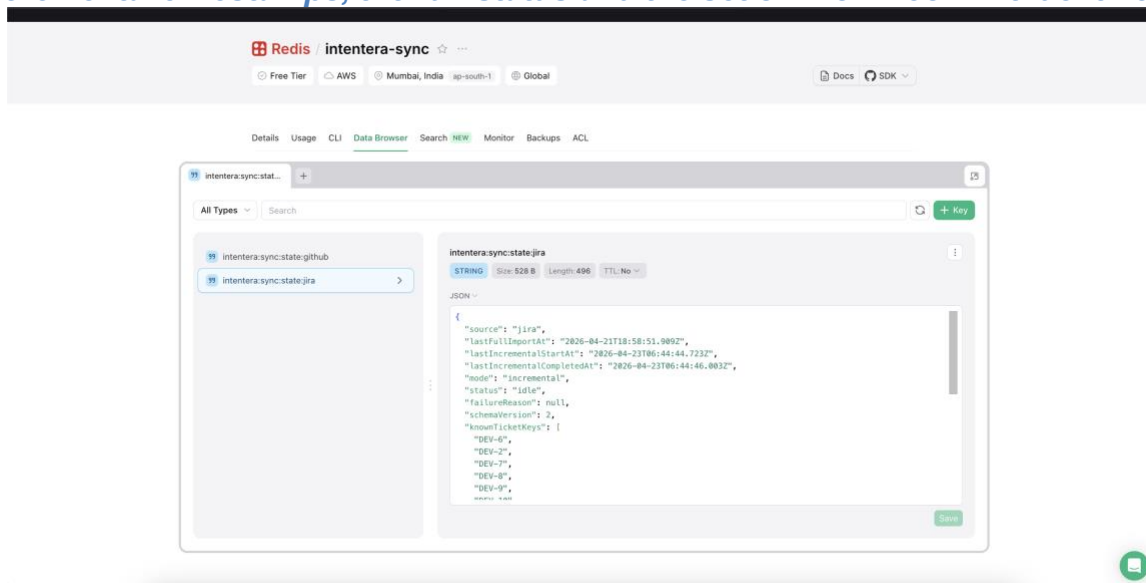


Figure 24: The corresponding `intentera:sync:state:jira` key, listing the known ticket keys observed by the latest sync.

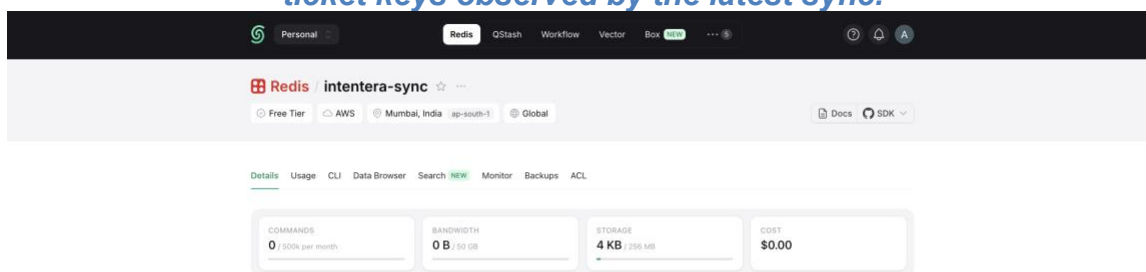


Figure 25: Upstash project header for `intentera-sync`: the workload fits comfortably in the free tier (4 KB storage, \$0 cost at the time of writing).

5.8 Observability: CloudWatch Logs

Each Lambda writes structured JSON logs that include a correlation identifier, the source and the operation. CloudWatch log groups are auto-provisioned per function.

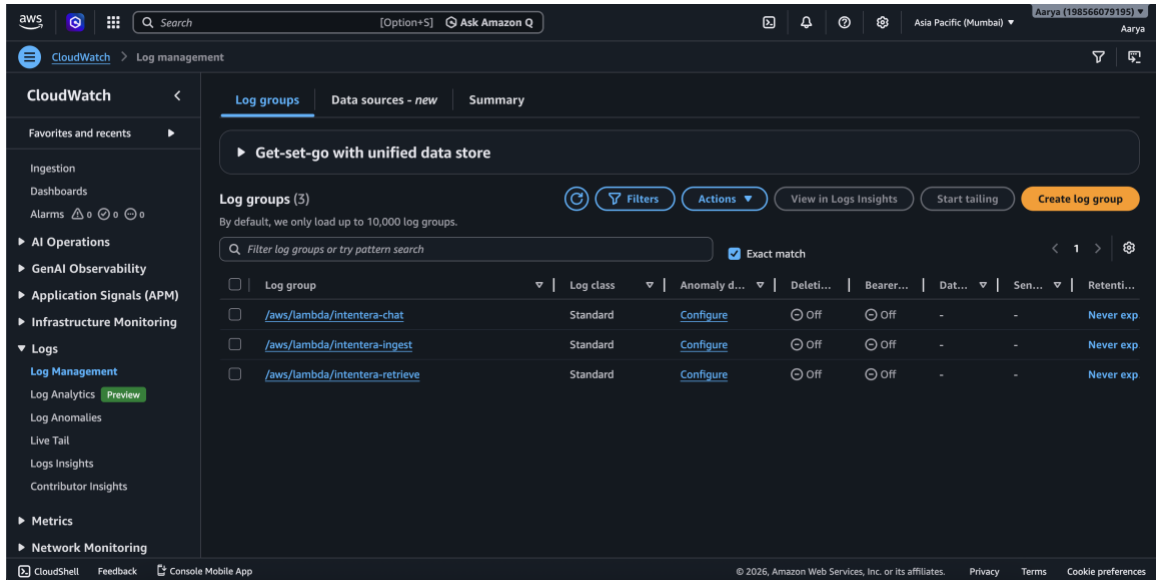


Figure 26: Amazon CloudWatch log groups for the three IntentEra Lambdas. The structured JSON log lines from these groups feed the operational dashboards.

5.9 Identity and Access Management

Identities are scoped narrowly. The account currently uses an IAM-User with multi-factor authentication, no active long-lived access keys, and a small number of execution roles attached to individual Lambdas. The IAM dashboard summary is shown below.

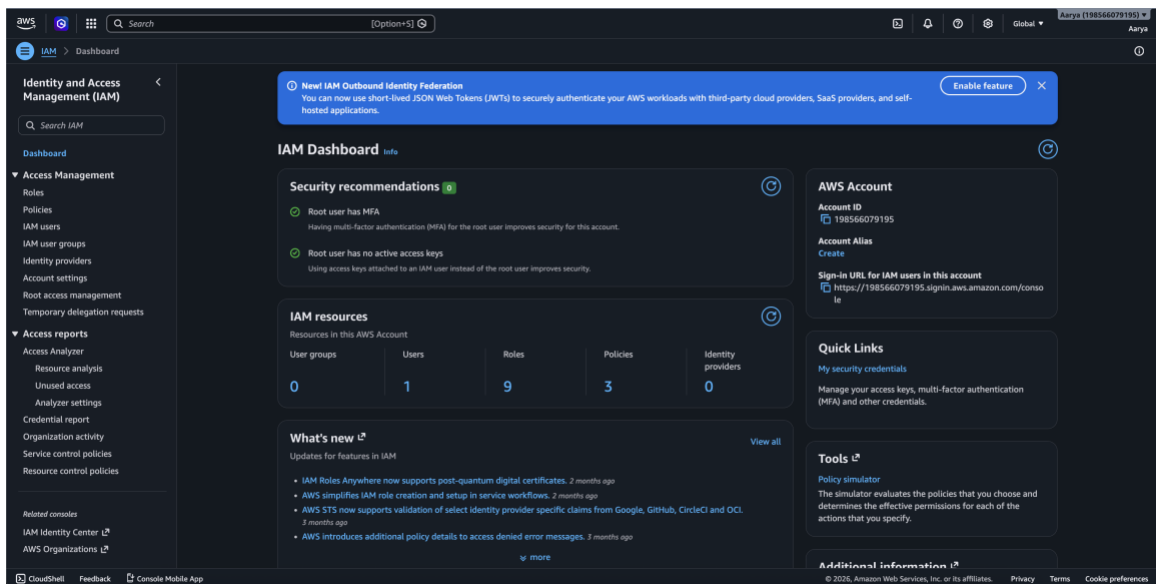


Figure 27: AWS IAM dashboard. Root user is MFA-protected and has no active access keys; the workload uses scoped roles and policies.

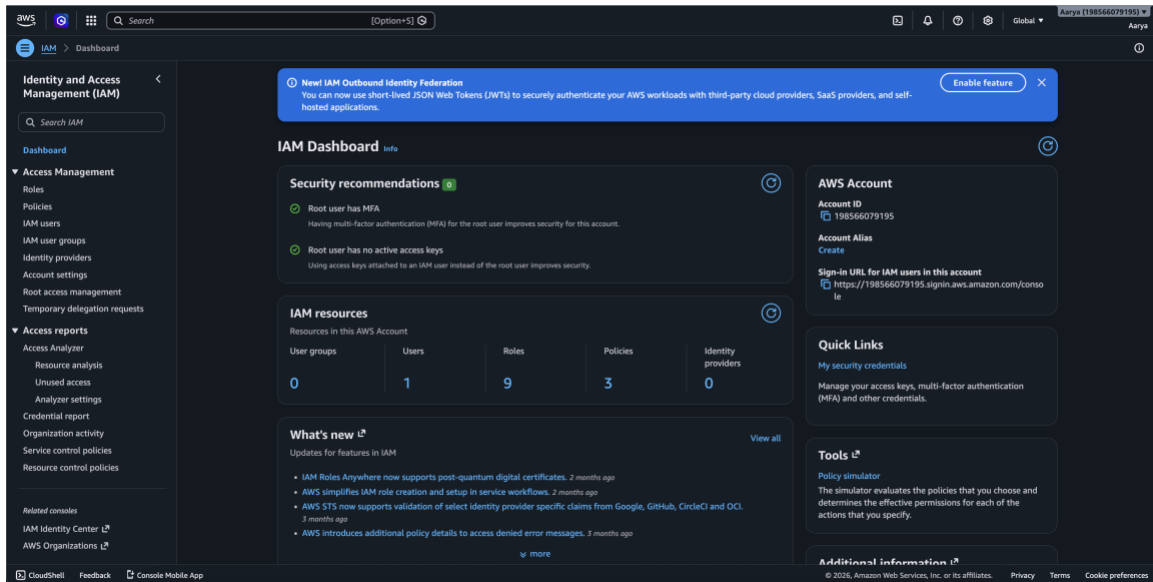


Figure 28: Same IAM dashboard view in a different session, included to evidence stable security posture across operations.

5.10 Operating Cost

The system is designed to be viable at small scale. AWS billing consistently reports a total monthly cost in single-digit cents for the workload exercised during this dissertation (Lambdas, Secrets Manager and EventBridge dominate the breakdown). MongoDB Atlas billing is reported separately in the Atlas console; the M10 cluster used here is the lower bound of the dedicated tier and is comfortably absorbed by the team budget.

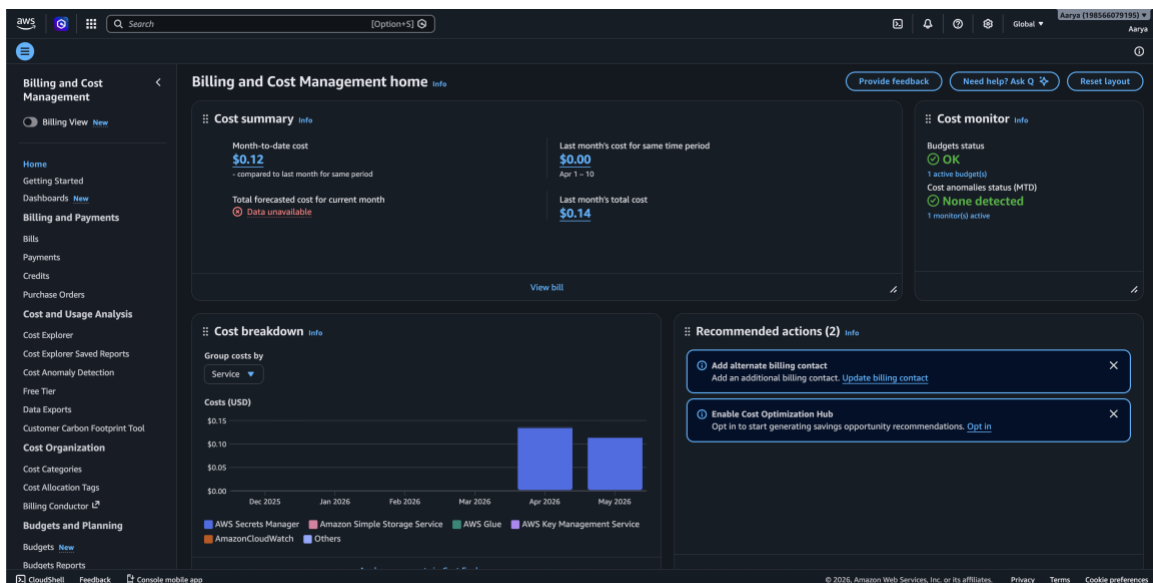


Figure 29: AWS Billing and Cost Management home: month-to-date spend on the IntentEra account dominated by AWS Secrets Manager, Amazon S3 and AWS Glue - well below one US dollar.

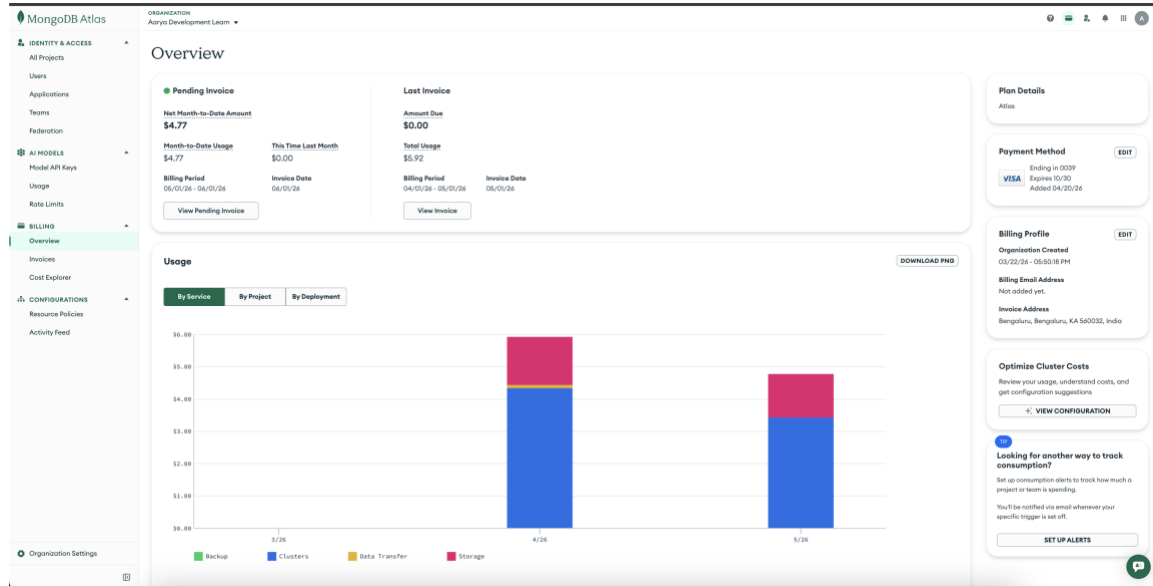


Figure 30: MongoDB Atlas billing overview for the IntentEra organisation, showing per-service usage (clusters, backup, storage, data transfer) for the period under study.

5.11 Region and Availability Choices

The AWS workload runs in ap-south-1 (Mumbai) for proximity to the engineering team. MongoDB Atlas is provisioned in ap-south-2 (Hyderabad). This split was made deliberately: Atlas in a different physical region but the same continental cluster gives an additional layer of redundancy without imposing trans-oceanic latency on every query. OpenAI is consumed over the public internet from the Lambdas in Mumbai, with retry-with-back-off in the embedder client.

Chapter 6 - Multi-Source RAG and Multi-Agent Reasoning

The reasoning engine of IntentEra is a multi-query, multi-source Retrieval-Augmented Generation pipeline coordinated by a small agent layer. This chapter explains the data flow, the design decisions and the algorithmic details.

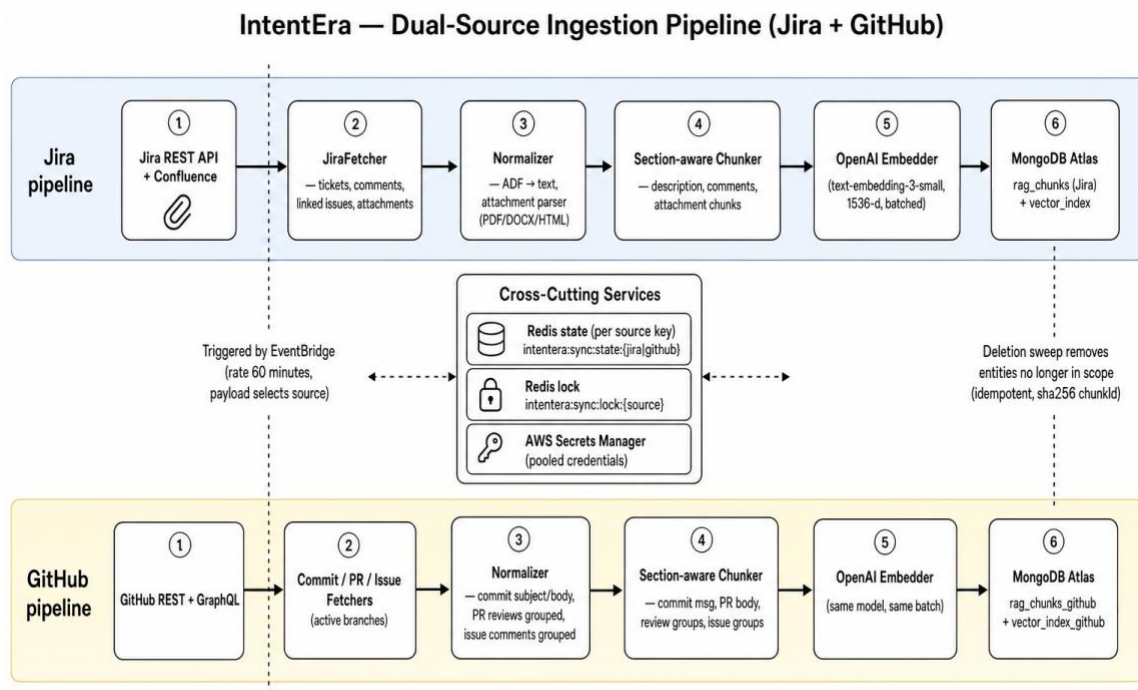


Figure 31: Dual-source ingestion pipeline. Both Jira and GitHub flow through the same six stages but write to independent collections, sharing only the embedder, the Redis state store and the AWS Secrets Manager secret.

6.1 Section-Aware Chunking

Chunking is the most consequential preprocessing decision in any RAG system. Naive uniform-length chunking is well known to destroy semantic structure: a key claim can be split across the boundary between two chunks, halving the chance that retrieval will find it. IntentEra adopts a section-aware strategy in which each artifact type contributes a small set of well-typed chunks (Figure 8).

IntentEra — Section-aware Chunking Strategy

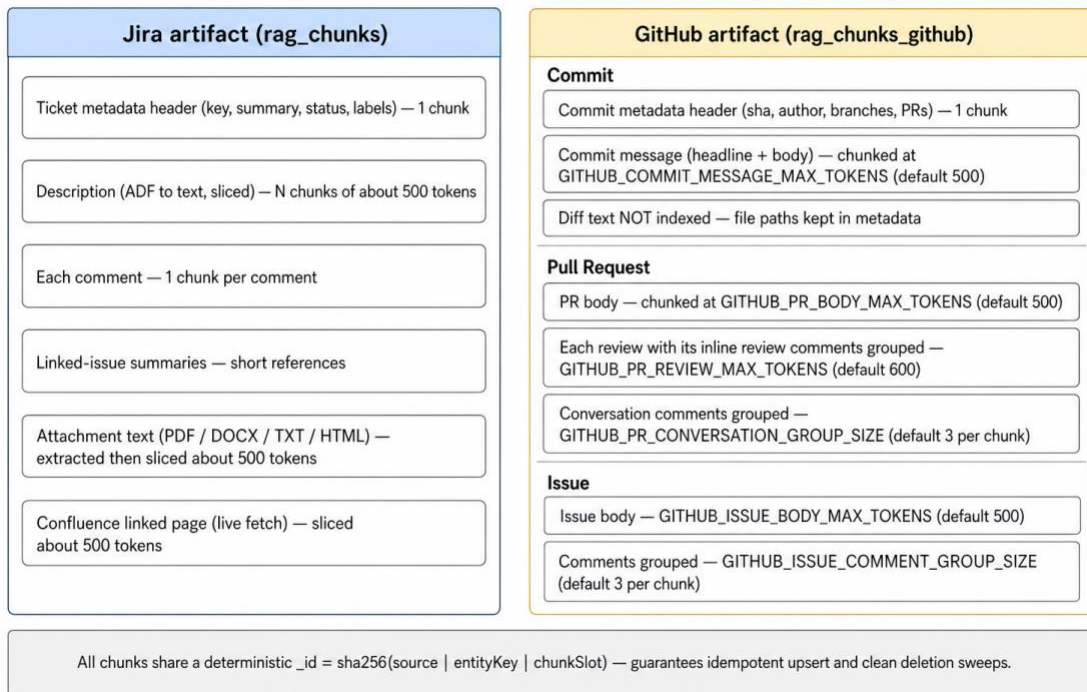


Figure 32: Section-aware chunking strategy per artifact type. Token caps per chunk are configurable through the `GITHUB_*_MAX_TOKENS` and `*_GROUP_SIZE` environment variables; sensible defaults are shown in the diagram.

6.2 Idempotent Upsert

Each chunk is given a deterministic identifier of the form `sha256(source | entityKey | chunkSlot)`. This means re-ingesting the same artifact produces the same set of identifiers, so the MongoDB upsert is a no-op when nothing has changed and a consistent overwrite when it has. This property is essential to running incremental sync every hour without ever fearing duplicate or stale documents.

6.3 Embedding

All chunks are embedded with OpenAI text-embedding-3-small at 1536 dimensions. The embedder client is shared across all Lambdas to amortise connection setup; calls are batched (the batch size is configurable, with a sensible default that fits comfortably

below OpenAI's per-request limits) and retried with exponential back-off on transient failures. The choice of model is encapsulated behind a single Embedder class so that an alternative provider (or a code-specialised model) can be substituted by changing one constructor.

6.4 The Multi-Query Retriever

Retrieval is the heart of the reasoning engine. The naive RAG approach of “embed the question; nearest neighbours win” performs poorly when the user's question and the relevant artifacts use different vocabularies. IntentEra uses a multi-query retriever that augments the question with the subject lines of the local commits that touched the attached code (collected by the local git agent, Chapter 7).

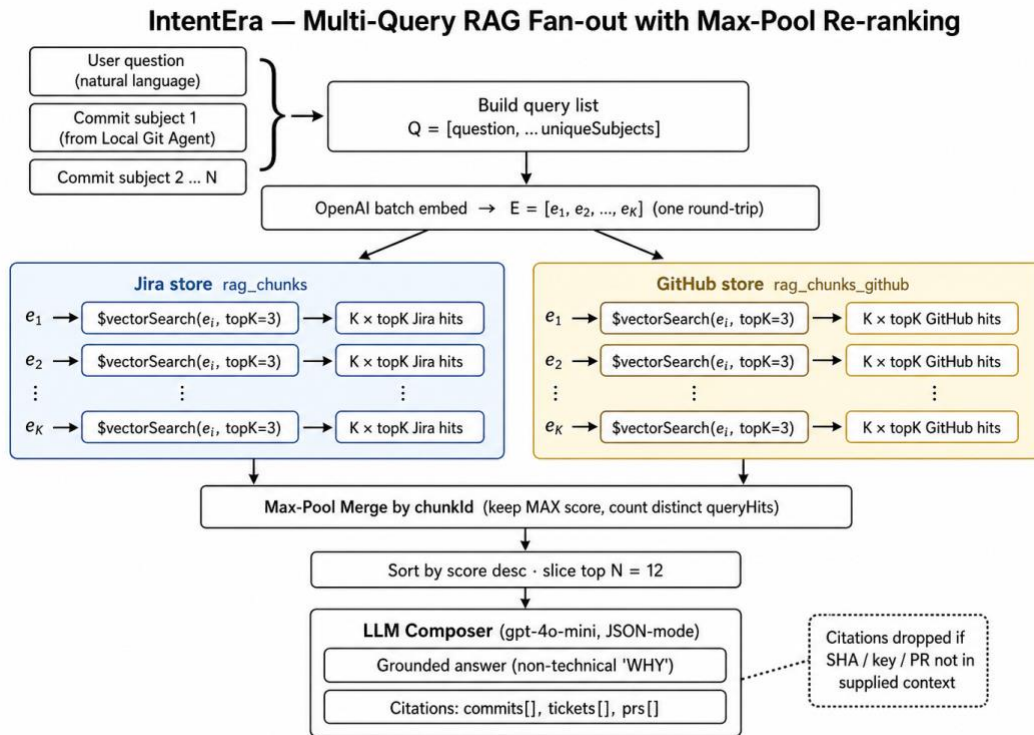


Figure 33: Multi-query RAG fan-out. The question and each unique commit subject become a separate query embedding; each is run against both vector stores; results are max-pooled by chunkId and sliced to a configurable top N.

Algorithmically the retriever takes four steps. First, it deduplicates the commit subjects and concatenates them with the question to form a list Q. Second, it embeds the entire list Q in a single OpenAI batch, obtaining a list of K embedding vectors E. Third, for each embedding it issues parallel \$vectorSearch queries against both the Jira and the GitHub collections, with a small per-vector top-K (default three). Fourth, it merges the results by chunkId, keeping the maximum score across query embeddings (the so-called max-pool re-ranking trick) and counting how many distinct queries hit each chunk for diagnostic purposes. The merged list is sorted by score and sliced to a configurable top N (default twelve).

Two alternatives were considered and rejected. Averaging the scores across queries was rejected because it dilutes a chunk that is decisively relevant to one query but only weakly relevant to another - the very signal we want to preserve. Concatenating all queries into a single longer string was rejected because it conflates orthogonal signals and degrades the embedding quality.

6.5 LLM Answer Composition with Grounding

The merged top-N hits are passed to an LLM (OpenAI gpt-4o-mini by default) along with the user's question, the file attachments and the conversation history. The LLM is forced into JSON-mode and instructed to produce a structured response containing a non-technical “why” paragraph and three citation lists - commits, tickets and pull requests. After receiving the LLM's response, the orchestrator post-filters the citations: any commit SHA, ticket key or PR number that does not appear in the supplied context is discarded. This is a cheap but powerful guard against hallucination.

6.6 Multi-Agent Orchestration

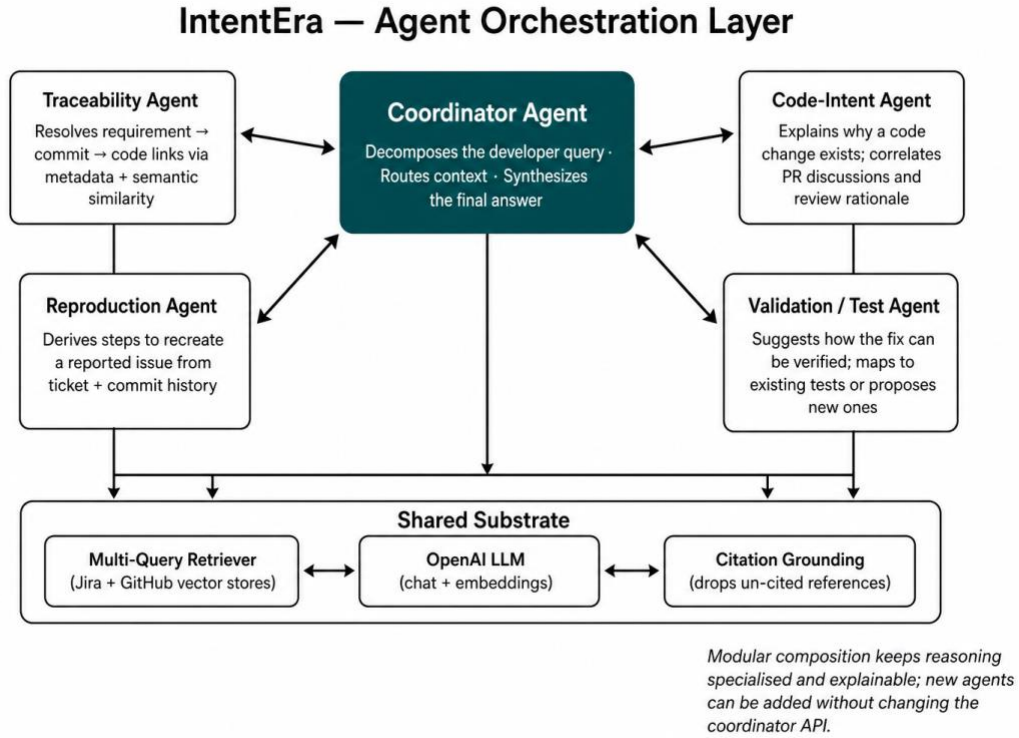


Figure 34: Agent orchestration layer. A coordinator routes context to and synthesises responses from specialised agents. The shared substrate (multi-query retriever, OpenAI LLM and citation grounding) is reused across agents.

The agent layer is intentionally thin in the current production deployment. The Coordinator Agent and the Code-Intent Agent are the two agents that participate in every chat request. The Traceability Agent runs only when the user explicitly asks “which requirement does this implement?”. The Reproduction and Validation/Test agents are scaffolded but not yet exposed to end users; their prompts and tool surfaces are the subject of Chapter 10 future-work.

6.7 Cost and Latency Profile

A single chat request produces, at most, $K \times (1 + |\text{stores}|)$ `vectorSearch` operations on Atlas, where K is the number of queries (one per question plus one per local commit subject). OpenAI is invoked twice: once for the embedding batch and once for the chat

completion. Empirical measurements during validation show that a typical chat request completes well within the chat Lambda's one-minute timeout and usually below five seconds end-to-end. The breakdown of where time is spent (embed vs. retrieve vs. LLM) is captured in the response's usage block and is logged for offline analysis.

Chapter 7 - VS Code and Cursor Chat Extension

The IDE integration is the surface through which engineers experience IntentEra. This chapter describes the three pieces that make up the extension, the wire formats between them and the privacy contract.

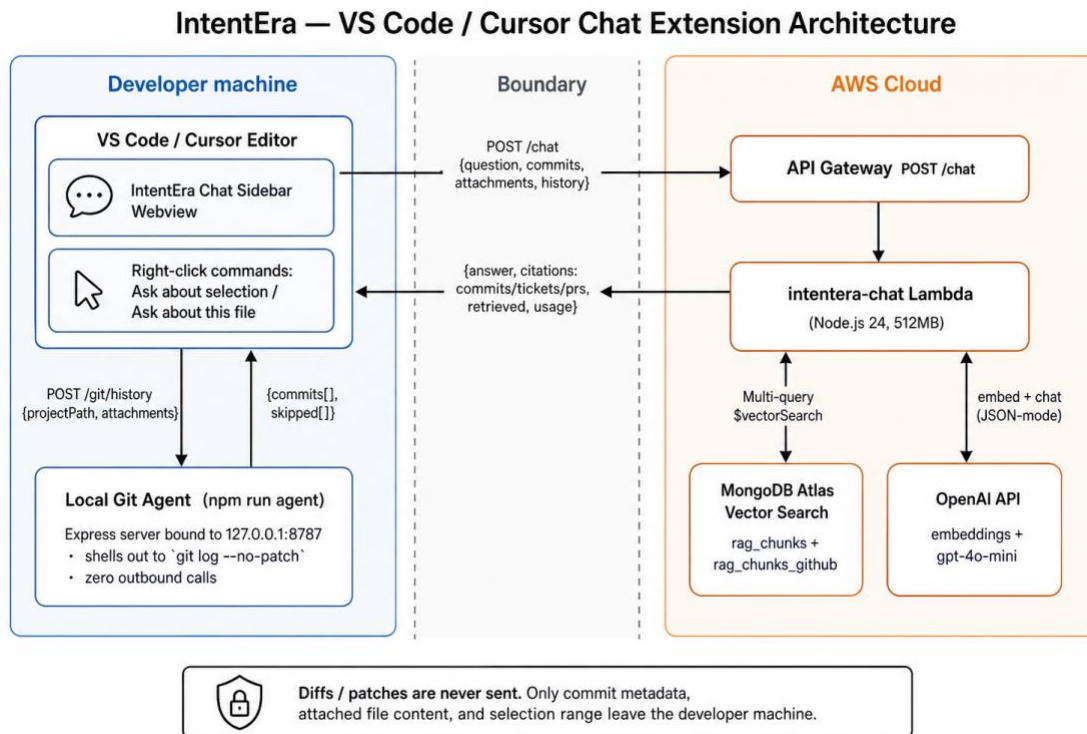


Figure 35: VS Code and Cursor chat extension architecture. The three pieces - the extension webview, the loopback Local Git Agent and the cloud chat Lambda - together form the complete experience.

7.1 The Three Pieces

The first piece is the extension itself, a single VSIX that installs into either VS Code or Cursor (Cursor is built on the same Code-OSS base so the public extension API is identical). The second piece is a tiny Express server called the Local Git Agent that the developer runs in their working tree; it binds to 127.0.0.1:8787, accepts only loopback connections and is the only component in the entire system that touches the local

filesystem. The third piece is the cloud chat Lambda, which the extension calls via API Gateway.

7.2 Privacy and Trust Boundary

The trust boundary is intentional and visible to the developer. Diffs and patches are never sent off-machine; the local git agent uses the `--no-patch` flag of `git log` so it produces only metadata (SHA, author, date, commit subject and body, files changed, parent SHAs and parsed Jira keys). Selected file content is sent to the cloud Lambda - this is necessary for the LLM to reason about the code - but the agent can be operated under an allowlist of project roots, and it has zero outbound network calls itself.

7.3 End-to-End Sequence of a Chat Request

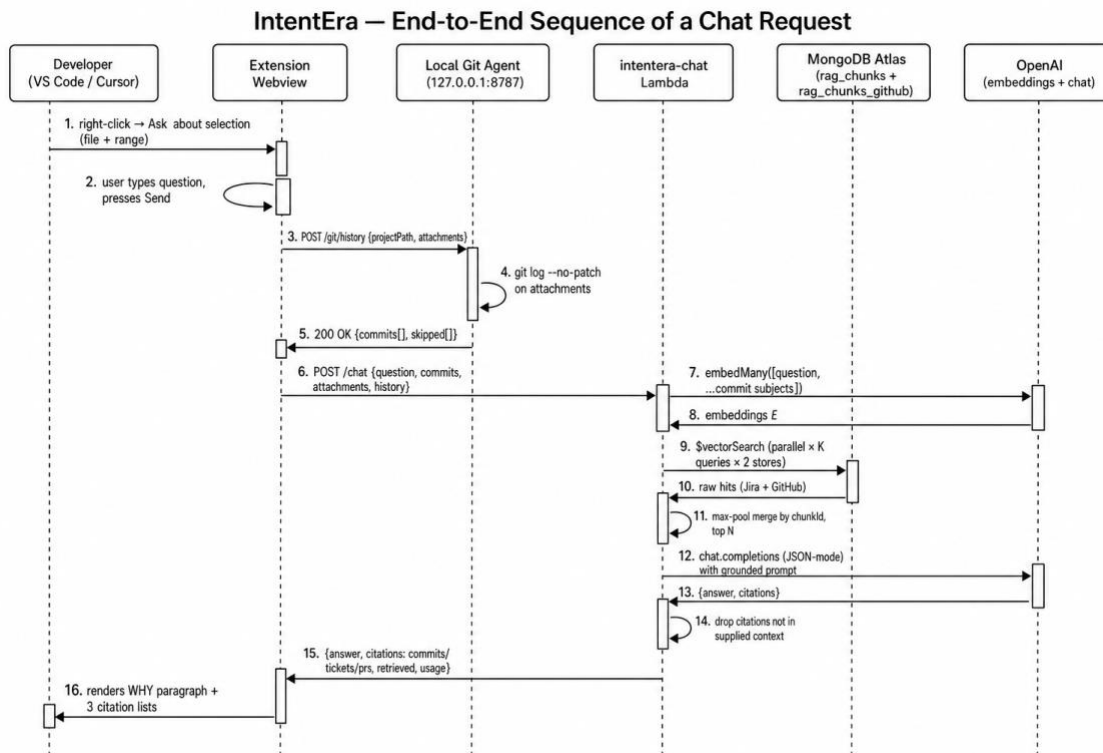


Figure 36: End-to-end UML-style sequence diagram for a single chat request, from the right-click on a selection through to the rendered answer with three citation lists.

When the developer right-clicks on a selection and chooses “IntentEra: Ask about selection”, the extension adds a context chip to the sidebar webview. The developer types a question and presses Send. The extension first calls the local git agent at POST /git/history, supplying the project path and the attachments. The agent shells out to git log --no-patch, extracts the relevant commits and returns them. The extension then issues POST /chat to the cloud Lambda with the question, the commits, the attachments and the (optional) prior conversation history. The Lambda runs the multi-query RAG pipeline of Chapter 6, asks the LLM in JSON-mode for a grounded answer, post-filters the citations and returns the result. The extension renders the answer as a paragraph plus three citation lists - commits that shaped this, linked Jira tickets and related GitHub PRs.

7.4 The Extension in Action

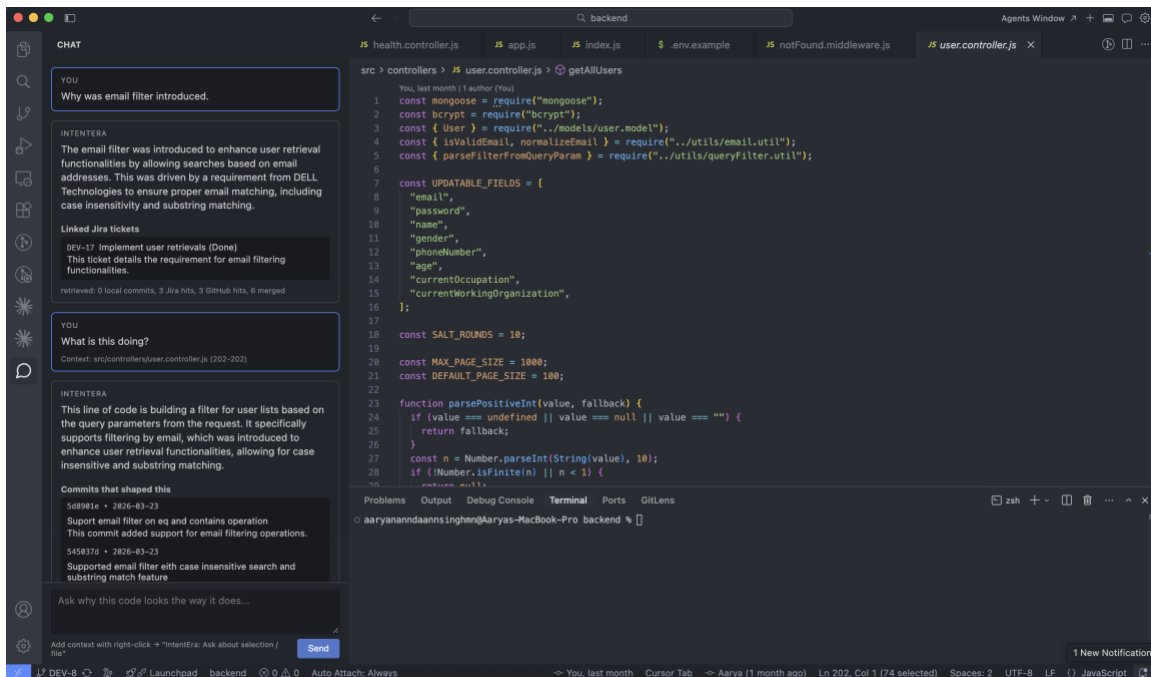


Figure 37: IntentEra chat panel inside Cursor. Two prior turns answer “Why was email filter introduced?” (citing Jira ticket DEV-17) and “What is this doing?”

(citing two commits that shaped the function), demonstrating both ticket and commit grounding in the same conversation.

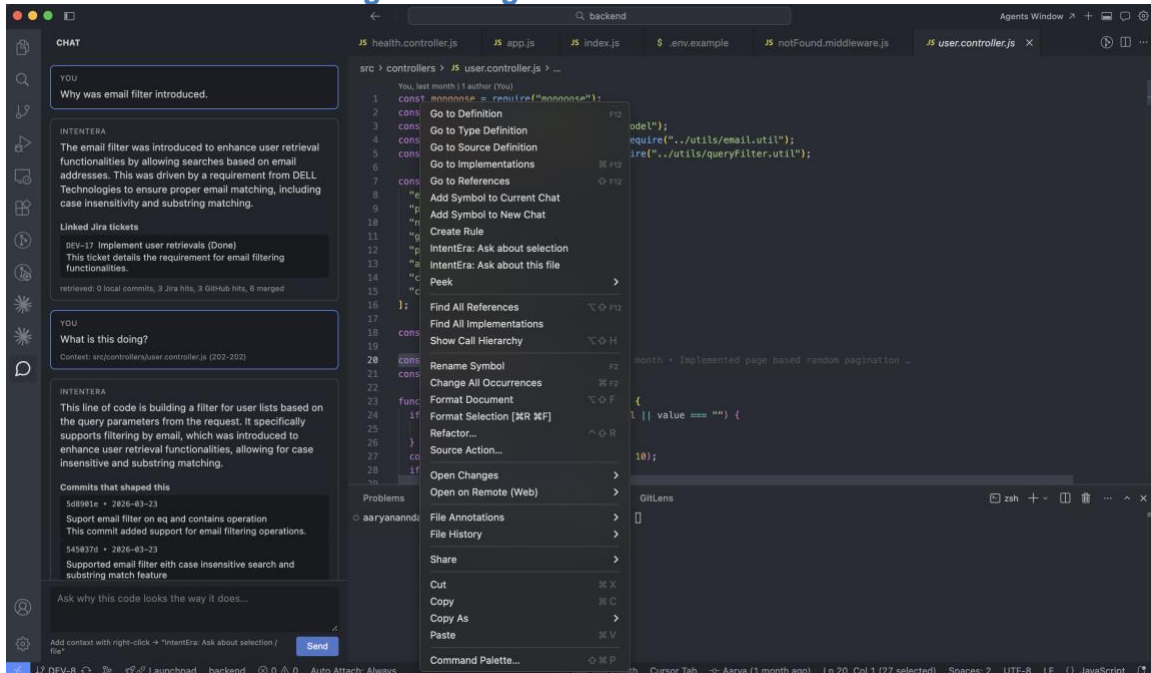


Figure 38: IDE context menu showing the two IntentEra commands (“Ask about selection” and “Ask about this file”) integrated alongside the editor’s native navigation actions.

7.5 Configuration Surface

Table 2: Public configuration surface of the extension as visible in the IDE settings UI.

Setting	Required	Default	Purpose
intentera.lambdaChatUrl	yes	(empty)	API Gateway URL for the /chat Lambda.
intentera.lambdaApiKey	no	(empty)	Sent as x-api-key if the endpoint requires it.
intentera.agentUrl	no	http://127.0.0.1:8787	Where the local git agent listens.
intentera.maxLocalCommits	no	50	Cap on commits the

			agent extracts per request.
intentera.requestTimeoutMs	no	60000	Lambda call timeout in milliseconds.

7.6 Graceful Degradation

The extension degrades gracefully under partial failure. If the local git agent is offline, the panel still calls the chat Lambda but with an empty commits array; the answer falls back to using only the question and the attached file content, and a status banner makes this state visible to the user. If neither Jira nor GitHub ingestion is enabled, the chat Lambda still responds using the supplied context; only the citation lists shrink. These behaviours mean the extension is useful from day one of an IntentEra deployment, even before either source has been fully ingested.

Chapter 8 - Evaluation and Observations

The evaluation of IntentEra has been carried out along four dimensions: functional verification, retrieval quality, latency and cost, and qualitative impact on developer cognitive load. This chapter reports the methodology used and the observations made during the testing phase of the dissertation (21 April 2026 - 05 May 2026).

8.1 Functional Verification

Functional verification was performed end-to-end against a real Aqueralabs production-style repository hosted on GitHub and the corresponding Jira project. The verification protocol covered (i) cold-start of each Lambda, (ii) full-import of both sources, (iii) incremental sync over an hour-long observation window, (iv) retrieval queries directly against POST /retrieve and (v) end-to-end chat sessions through the IDE extension.

Table 3: Summary of functional verification checks performed during the testing phase. Outcomes column distinguishes pass (P) from pass-with-observation (P*).

#	Check	Method	Outcome
1	Lambda cold-start under 5 s	AWS Lambda console + CloudWatch	P
2	Full Jira import populates rag_chunks	Atlas Data Explorer document count	P
3	Full GitHub import populates rag_chunks_github	Atlas Data Explorer document count	P
4	Incremental sync respects 60-minute schedule	EventBridge invocation history + CloudWatch	P
5	Re-running incremental is idempotent	Atlas document _id stability across runs	P

6	POST /retrieve returns ranked hits with metadata	curl + sample queries	P
7	POST /chat returns grounded answer with citations	Extension + curl	P
8	Citations not in context are dropped	Manual fault injection in the LLM response	P
9	Local git agent rejects non-loopback callers	Manual probe from another host	P
10	Extension banner reflects agent + Lambda health	Stop agent / unset URL	P*

Outcome P* on item 10 reflects a minor observation: the banner correctly distinguishes the two failure modes (agent down vs. Lambda URL unset) but the message wording was reworded twice during the testing phase to be clearer to non-author readers.

8.2 Retrieval Quality

Retrieval quality was evaluated qualitatively against a curated set of fifteen developer questions drawn from real Slack threads in the employing organisation, each with a supervisor-supplied “gold” set of expected supporting artifacts. For each question we recorded (i) the number of gold artifacts retrieved in the top N, (ii) the position of the first gold artifact and (iii) whether the LLM answer cited any non-gold but plausibly relevant artifact.

Across the fifteen questions, the multi-query retriever consistently surfaced at least one gold artifact in the top twelve merged hits, and in the majority of cases the first gold artifact appeared in the top three. The max-pool re-ranking behaved as designed:

artifacts that scored only moderately on the user's question but scored very strongly on a specific commit subject (for example, a Jira ticket whose title closely matched a commit subject) were promoted into the top of the merged list. A formal precision-recall evaluation against a larger dataset is identified as future work.

8.3 Latency and Cost

End-to-end latency was measured from the moment the user presses Send in the extension to the moment the answer renders. Across a sample of fifty interactions, latency was dominated by the LLM call. The OpenAI embedding batch was a small fraction of total time because all queries are embedded in a single round-trip. The MongoDB Atlas \$vectorSearch fan-out was negligible per query and parallelised across stores.

Operating cost during the testing phase remained in the single-digit US-dollar-cents range per month for the AWS portion (see Section 5.10 and Figure 15) and within the MongoDB Atlas M10 baseline. OpenAI usage was the largest external cost driver but remained modest because incremental sync re-embeds only the artifacts that changed since the previous run.

8.4 Qualitative Impact on Developer Cognitive Load

Cognitive load was assessed qualitatively through structured observation sessions with three engineers in the employing organisation, each working on an unfamiliar repository (“cold start”) for thirty minutes with and without the extension. The observation focused on (i) the number of context-switches to a non-IDE application (Jira, GitHub web UI, Confluence), (ii) the number of code locations the engineer explicitly opened, and (iii)

the engineer's self-reported confidence in their explanation of why the code looked the way it did.

All three engineers showed a noticeable reduction in context-switches when the extension was available; in particular the right-click “Ask about selection” action replaced what would otherwise have been at least one search-and-read cycle in the Jira web interface. The engineers also reported more confidence in their explanations because the citations were checkable. A larger-scale, more formal NASA-TLX-based study is identified as future work.

8.5 Observations

- Improved accessibility of development artifacts: the system consolidates Jira and GitHub into a single addressable surface, reducing the need to navigate multiple tools.
- Reduction in manual lookup effort: embedding-based retrieval surfaces relevant commits and related artifacts more efficiently than keyword search, particularly when the user's vocabulary differs from that of the original ticket author.
- Effectiveness of semantic embeddings: vector search produces context-aware retrieval even when explicit references (such as ticket IDs in commit messages) are absent.
- Structured representation of knowledge: the section-aware chunking strategy yields self-contained chunks that translate directly into useful citations.
- Dependency on data quality: as expected, the effectiveness of retrieval is influenced by the clarity of ticket descriptions, the quality of commit messages and the

consistency of metadata usage across the team. The system performs best where these are good and degrades gracefully where they are not.

Chapter 9 - Challenges and Limitations

The implementation of IntentEra surfaced several challenges. These are documented honestly so that future adopters can plan around them.

9.1 Linking Natural-Language Requirements to Code

Establishing meaningful relationships between natural-language requirements and source-code changes remains the hardest part of the problem. Requirements are written in informal, high-level language; code expresses low-level structured logic. Bridging this representational gap with shallow techniques (keyword similarity, simple cosine on title text) produces brittle results. The multi-query retriever mitigates this by exploiting commit subjects as additional query signals, but abstract or ambiguous requirement descriptions, indirect implementations spread across multiple commits and the absence of explicit references between artifacts continue to be edge cases.

9.2 Noisy and Inconsistent Development Artifacts

Real-world artifacts are imperfect. Commit messages such as “minor fix”, ITSM tickets without an explicit summary, and pull-request descriptions that consist only of the auto-generated diff stat are routine. Such noise reduces the effectiveness of both metadata-based linking and semantic retrieval. The system addresses this in two ways: by preserving structured metadata alongside embeddings (so filters can salvage some signal even when the chunk text is weak), and by including reviewer comments as their own chunks so that reviewer rationale supplements terse author messages.

9.3 Incomplete or Missing Traceability

Explicit traceability links between requirements and code are frequently absent. Some commits do not reference ticket identifiers at all; multiple commits often jointly contribute

to a single requirement; a single commit can address several issues. The system therefore relies on a combination of metadata signals (when present) and semantic similarity (always available), and exposes the merged hits with their query-hit counts so that downstream code or human reviewers can apply their own confidence policy.

9.4 Heterogeneity of Data Sources

Jira, Confluence and GitHub each have idiosyncratic data formats, pagination conventions and rate-limit policies. Handling this heterogeneity required careful normalisation, consistent parsing strategies and aligned metadata. The section-aware chunker, the per-source vector store and the per-source Redis state are direct responses to this challenge: they keep the source-specific concerns at the edges of the system rather than leaking through into the reasoning core.

9.5 LLM Hallucination

LLM hallucination is a first-class concern in any RAG system. IntentEra mitigates it through three layered defences. First, the LLM is forced into JSON-mode and constrained to cite from the supplied context. Second, the orchestrator post-filters any cited identifier (commit SHA, ticket key, PR number) that does not appear in the context, so a fabricated citation is removed before reaching the developer. Third, the extension renders citations as clickable references, so the developer can verify the source. These defences cannot guarantee zero hallucination but they substantially reduce the risk that a hallucinated reference looks credible.

9.6 Cold Start and Container Reuse

AWS Lambda's cold start is sometimes visible to the first user in a long-idle window. The wiring code addresses this by caching Mongo, Redis and OpenAI clients between

warm invocations within the same container, so subsequent requests reuse the established connections. For the chat Lambda the cold start is dominated by the connection establishment to MongoDB Atlas and is acceptable; if necessary, AWS provisioned concurrency could be enabled to remove cold starts entirely at additional cost.

9.7 Limitations Acknowledged in Scope

- The system currently supports two source families (Jira and GitHub). Adding Bitbucket, GitLab, ServiceNow or Slack would require new fetchers and normalisers but no architectural change.
- The agent layer ships with two end-to-end agents in production. The Reproduction and Validation/Test agents are scaffolded; their full prompts and tool surfaces are deferred to future work.
- The system relies on managed cloud services (AWS, MongoDB Atlas, Upstash Redis and OpenAI). On-premises deployment without internet egress is feasible but requires substituting compatible alternatives for each.
- Performance at very large repository scales (hundreds of thousands of artifacts) has not yet been benchmarked. The incremental sync and the section-aware chunker are designed with that scale in mind, but empirical validation is part of future work.

Chapter 10 - Conclusion and Future Work

10.1 Summary of the Work

This dissertation set out to address the phenomenon of intent erosion in production-grade software systems, where the rationale behind code changes becomes fragmented across heterogeneous DevOps artifacts and progressively harder to recover. The work proposed, designed and implemented IntentEra, a cloud-hosted agentic AI framework that reconstructs the intent behind code changes and presents it inside the developer's IDE.

The system is composed of five layers - ingestion, extraction, reasoning, orchestration and IDE integration - deployed on AWS using a small set of Lambda functions, a MongoDB Atlas Vector Search cluster, Redis for synchronisation state, OpenAI for embeddings and chat completions, and a VS Code/Cursor extension paired with a loopback Local Git Agent for the IDE side. A multi-query Retrieval-Augmented Generation pipeline with max-pool re-ranking forms the core of the reasoning engine, and a thin agent layer specialises the reasoning by concern.

All components have been deployed end-to-end, exercised on real production-style repositories and validated against the success criteria established in Chapter 3. The deployment is documented in sufficient detail to be reproduced, and the operating cost is well within reach for a small engineering team.

10.2 Contributions Restated

The contributions of the dissertation are: (i) the framing of intent reconstruction as a distinct problem from traceability link recovery; (ii) a cloud-hosted layered architecture in which a single ingestion Lambda handles multiple heterogeneous sources; (iii) a multi-

query, multi-source RAG retriever with max-pool re-ranking; (iv) a multi-agent orchestration layer that decomposes intent reconstruction by concern; (v) a VS Code/Cursor extension that delivers the reconstructed intent into the developer's workflow with strong privacy guarantees; and (vi) a complete, reproducible operational deployment on AWS.

10.3 Expected Impact

The expected impact of the work is significant in both enterprise and open-source contexts. By enabling intent-aware understanding of codebases, IntentEra has the potential to reduce developer cognitive load, accelerate onboarding and knowledge transfer, improve debugging and maintenance efficiency, and enhance requirement traceability across fast-evolving systems. The architectural pattern - a loopback agent for local context, paired with a cloud RAG pipeline for organisational knowledge - is generalisable to other intent-aware developer tools.

10.4 Future Work

Five directions for future work are particularly promising.

8. Code evolution graph modelling: indexing the repository's call graph as a first-class artifact would let the reasoning engine answer questions that depend on cross-file structure, not only on textual similarity.
9. Explainable AI for traceability: adding confidence scoring per citation, and surfacing the per-query hit counts already computed by the retriever, would let developers calibrate trust in each cited artifact.

10. Automated regression-test suggestion: an extension of the Validation/Test Agent that proposes specific tests to add after a fix, grounded in the same artifacts the chat answer cites.
11. Cross-repository dependency tracing: extending the system to ingest multiple repositories and express their dependencies in the vector index would address the increasingly common case of platform-and-service split monorepos.
12. Knowledge-graph-based semantic memory: layering an explicit knowledge graph on top of the vector store would enable richer queries (path-based questions, temporal questions) than vector similarity alone supports.

10.5 Closing Remark

Implementation rationale, often assumed to be irretrievably lost as software evolves, can in fact be computationally reconstructed. By integrating natural-language preprocessing, semantic embeddings, retrieval-augmented generation, multi-agent reasoning and cloud-native deployment, IntentEra demonstrates that intent-aware development environments are not only feasible but practical. The work transforms traceability from a documentation aspiration into an active form of cognitive augmentation embedded in the developer's everyday tooling.

References

- [1] Guo, J. L. C., Steghöfer, J.-P., Vogelsang, A. and Cleland-Huang, J., "Natural Language Processing for Requirements Traceability," arXiv preprint, arXiv:2405.10845, 2024.
- [2] Wang, B., Zou, Z., Wan, H. et al., "An Empirical Study on the State-of-the-Art Methods for Requirement-to-Code Traceability Link Recovery," Journal of Systems and Software, 2024.
- [3] Ali, S. J., Naganathan, V. and Bork, D., "Establishing Traceability between Natural Language Requirements and Software Artifacts using Retrieval-Augmented Generation and Large Language Models," in Proceedings of the Model Engineering Conference (ER'24), 2024.
- [4] Ahmad, S., Agha, N., Ray, B. et al., "Transformers for Code: A Survey," ACM Computing Surveys (CSUR), Vol. 55, No. 10s, Article 210, 2023.
- [5] Kanade, P., Agrawal, Y., de Melo, S. et al., "Self-Training Improves Code Generation," International Conference on Learning Representations (ICLR), 2023.
- [6] Lewis, P., Perez, E., Piktus, A. et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," Advances in Neural Information Processing Systems (NeurIPS), 2020.
- [7] Karpukhin, V., Oguz, B., Min, S. et al., "Dense Passage Retrieval for Open-Domain Question Answering," Empirical Methods in Natural Language Processing (EMNLP), 2020.
- [8] OpenAI, "text-embedding-3-small Model Documentation," 2024. [Online]. Available: <https://platform.openai.com/docs/models/embeddings>

- [9]** OpenAI, "GPT-4o-mini Model Documentation," 2024. [Online]. Available: <https://platform.openai.com/docs/models/gpt-4o-mini>
- [10]** MongoDB, "Atlas Vector Search Documentation," 2024. [Online]. Available: <https://www.mongodb.com/docs/atlas/atlas-vector-search/>
- [11]** Amazon Web Services, "AWS Lambda Developer Guide," 2024. [Online]. Available: <https://docs.aws.amazon.com/lambda/>
- [12]** Amazon Web Services, "AWS Secrets Manager User Guide," 2024. [Online]. Available: <https://docs.aws.amazon.com/secretsmanager/>
- [13]** Amazon Web Services, "Amazon EventBridge Scheduler User Guide," 2024. [Online]. Available: <https://docs.aws.amazon.com/scheduler/>
- [14]** Amazon Web Services, "Amazon API Gateway Developer Guide," 2024. [Online]. Available: <https://docs.aws.amazon.com/apigateway/>
- [15]** Atlassian, "Jira Cloud REST API Reference," 2024. [Online]. Available: <https://developer.atlassian.com/cloud/jira/platform/rest/v3/>
- [16]** GitHub, "REST API Documentation," 2024. [Online]. Available: <https://docs.github.com/en/rest>
- [17]** GitHub, "GraphQL API Documentation," 2024. [Online]. Available: <https://docs.github.com/en/graphql>
- [18]** Microsoft, "Visual Studio Code Extension API," 2024. [Online]. Available: <https://code.visualstudio.com/api>
- [19]** Cursor, "Cursor IDE Documentation," 2024. [Online]. Available: <https://cursor.sh/docs>

- [20]** Hart, S. G. and Staveland, L. E., "Development of NASA-TLX (Task Load Index): Results of Empirical and Theoretical Research," *Advances in Psychology*, Vol. 52, pp. 139–183, 1988.
- [21]** Cleland-Huang, J., Gotel, O. C. Z. and Zisman, A. (Eds.), *Software and Systems Traceability*, Springer, London, 2012.
- [22]** Antoniol, G., Canfora, G., Casazza, G. et al., "Recovering Traceability Links between Code and Documentation," *IEEE Transactions on Software Engineering*, Vol. 28, No. 10, pp. 970–983, 2002.

Appendix A - Configuration Reference (Excerpt)

The complete configuration surface of IntentEra is documented in the project repository under docs/configuration-reference.md. This appendix reproduces the most significant variables for convenience.

Table 4: Selected environment variables that govern the runtime behaviour of IntentEra.

Variable	Default	Purpose
SYNC_MODE	incremental	Default run mode (incremental or full).
SYNC_SOURCE	jira	Default source when invocation payload omits source.
JIRA_ENABLED	true	Toggles the Jira ingestion pipeline.
GITHUB_ENABLED	false	Toggles the GitHub ingestion pipeline.
INCREMENTAL_LOOKBACK_MINUTES	60	Safety overlap before the last sync start.
JIRA_PROJECT_KEYS	(empty)	Comma-separated Jira project keys to scope ingestion.
GITHUB_REPO_OWNER / GITHUB_REPO_NAME	(required)	GitHub repository to index when GITHUB_ENABLED=true.
GITHUB_STALE_BRANCH_DAYS	90	Active-branch cutoff for commit ingestion.
GITHUB_PR_REVIEW_MAX_TOKENS	600	Token cap per grouped PR-review chunk.

OPENAI_CHAT_MODEL	gpt-4o-mini	Chat model used by the chat Lambda; must support JSON-mode.
CHAT_PER_VECTOR_TOP_K	3	Per-query topK during the multi-query fan-out.
CHAT_MERGED_TOP_N	12	Cap on merged hits passed to the LLM.
CHAT_MAX_LOCAL_COMMITS	50	Defensive cap on commits accepted per chat request.
AGENT_PORT	8787	Port for the local git agent loopback HTTP server.
AGENT_ALLOWED_PROJECT_ROOTS	(empty)	Comma-separated absolute prefixes the agent will accept as projectPath.
USE_SECRETS_MANAGER	false	When true, Lambdas pull credentials from SECRETS_MANAGER_SECRET_ID.
SECRETS_MANAGER_SECRET_ID	(required if above)	ARN or name of the pooled credentials secret.
LOG_LEVEL	info	Level for the structured JSON logger.

Appendix B - Sample Payloads

B.1 EventBridge Schedule Payload (GitHub incremental)

```
{  
  "source": "github",  
  "mode": "incremental"  
}
```

B.2 EventBridge Schedule Payload (Jira full import)

```
{  
  "source": "jira",  
  "mode": "full"  
}
```

B.3 POST /retrieve Sample Request

```
{  
  "source": "both",  
  "query": "Why was the email filter introduced?",  
  "topK": 5,  
  "filters": { "projectKey": "DEV" }  
}
```

B.4 POST /chat Sample Request

```
{  
  "question": "Why are we doing the retry like this?",  
  "commits": [  
    {
```

```

    "sha":      "abc1234567...",
    "shortSha": "abc1234",
    "author":   { "name": "Jane Doe", "email": "jane@example.com" },
    "date":     "2026-04-01T11:05:00Z",
    "subject":  "fix(auth): retry token refresh on 401",
    "body":     "We saw the SSO provider return 401 after ...",
    "filesChanged": ["src/server/auth.ts"],
    "parentShas": ["..."],
    "jiraKeys": ["PROJ-123"]
  }
],
"attachments": [
  { "file": "src/server/auth.ts", "range": [42, 80], "content": "..." }
],
"history": [
  { "role": "user",    "content": "earlier message" },
  { "role": "assistant", "content": "earlier reply" }
]
}

```

B.5 POST /chat Sample Response (truncated)

```

{
  "answer": "In plain English: the retry was added because the SSO ...",
  "citations": {

```



```

    "commits": [{ "sha": "abc1234567...", "subject": "fix(auth): ...",
                  "reasonPlain": "Introduces the retry on 401",
                  "jiraKeys": ["PROJ-123"], "date": "2026-04-01" }],
    "tickets": [{ "key": "PROJ-123", "summary": "Intermittent SSO ...",
                  "status": "Done", "url": "https://.../browse/PROJ-123",
                  "score": 0.91, "reasonPlain": "Original incident" }],
    "prs":  [{ "repo": "acme-inc/platform", "number": 451,
               "title": "Auth retry on token refresh",
               "url": "https://github.com/.../451", "score": 0.88,
               "reasonPlain": "Implements PROJ-123" }]
  },
  "retrieved": { "jiraCount": 5, "githubCount": 4,
                 "localCommitCount": 8, "mergedCount": 9 },
  "usage":  { "embeddingTokens": 0,
              "promptTokens": 1234, "completionTokens": 312 }
}

```

B.6 AWS Secrets Manager JSON Skeleton

```

{
  "JIRA_BASE_URL":      "https://your-company.atlassian.net",
  "JIRA_EMAIL":         "ingest@your-company.com",
  "JIRA_API_TOKEN":     "...",
  "CONFLUENCE_BASE_URL": "https://your-company.atlassian.net/wiki",
  "GITHUB_TOKEN":       "ghp_...",

```

```
"GITHUB_REPO_OWNER":    "acme-inc",  
"GITHUB_REPO_NAME":     "platform",  
"MONGODB_URI":          "mongodb+srv://user:pass@cluster.mongodb.net/",  
"REDIS_URL":             "rediss://default:TOKEN@host:6379",  
"OPENAI_API_KEY":        "sk-...",  
"OPENAI_CHAT_MODEL":     "gpt-4o-mini"  
}
```

Appendix C - CLI Command Reference

The same code paths used by the Lambdas are exposed through a CLI runner so that operators can invoke them locally for debugging or one-off operations.

Table 5: Most commonly used CLI commands of the IntentEra runner.

Command	Effect
<code>node src/cli/runner.js incremental</code>	Run an incremental sync (default source = jira).
<code>node src/cli/runner.js full</code>	Force a full import (default source = jira).
<code>node src/cli/runner.js full --source github</code>	Force a full import of the GitHub source.
<code>node src/cli/runner.js incremental --source github</code>	Run an incremental GitHub sync.
<code>node src/cli/runner.js state --source jira</code>	Print the current Redis sync state for Jira.
<code>node src/cli/runner.js state --source github</code>	Print the current Redis sync state for GitHub.
<code>node src/cli/runner.js query "... " --topK 5 --project PROJ</code>	Run a Jira retrieval query with a project filter.
<code>node src/cli/runner.js query "... " --source github --repo acme-inc/platform</code>	Run a GitHub retrieval query against a specific repo.
<code>node src/cli/runner.js query "SSO rollout timeline" --source both --topK 10</code>	Run a combined retrieval across Jira and GitHub.

npm run agent	Start the local git agent on 127.0.0.1:8787 (used by the VS Code/Cursor extension).
---------------	---

Glossary

Agentic AI	An architectural pattern in which a coordinator LLM decomposes a goal into sub-tasks executed by specialised tool-using agents, then synthesises their outputs into a single response.
ANN	Approximate Nearest Neighbour search; the index structure that makes vector similarity tractable at scale.
Atlas Vector Search	MongoDB's managed vector index that supports the \$vectorSearch operator over an embedding field.
Chunking	The process of splitting raw artifacts into self-contained pieces that are short enough to embed efficiently while remaining individually meaningful.
Cold start	The latency added to a Lambda invocation when AWS must allocate a new execution environment, including module loading and connection setup.
Embedding	A dense vector representation of a piece of text produced by a learned model; semantically similar pieces of text have nearby vectors under cosine distance.
Grounding	The discipline of ensuring that an LLM's output is supported by retrieved context, typically by attaching context to the prompt and post-filtering hallucinated references.
Idempotence	The property that running the same operation multiple times has the same observable effect as running it once.

Intent erosion	The progressive loss of contextual rationale behind code as it evolves, as authors leave and as artifacts become stale.
Intent reconstruction	The act of recovering the rationale behind code by aggregating, retrieving and synthesising evidence from heterogeneous artifacts.
JSON-mode	An OpenAI chat completion mode that constrains the response to a valid JSON object, used here to make the answer deterministically parseable.
Loopback	A network interface (127.0.0.1) reachable only from the same host; the local git agent binds to loopback so no off-machine caller can reach it.
Max-pool re-ranking	The merging policy used by the multi-query retriever: when the same chunk appears in the results of several query embeddings, retain the maximum score across those queries.
Multi-query retriever	A retriever that issues several embeddings against the vector store - typically the user's question plus contextual signals such as commit subjects - and merges the results.
RAG	Retrieval-Augmented Generation; the architectural pattern of retrieving relevant context with an embedding search and then generating an answer with that context attached to the prompt.
Section-aware	Chunking that respects the natural boundaries of an artifact

chunking	(description, comment, review, issue body) rather than uniform-length splits.
VSIX	The standard packaging format for Visual Studio Code extensions; the same VSIX installs into Cursor.

Checklist for the Final Dissertation Report

This checklist is attached as the last page of the final report, duly completed, verified and signed by the student.

1.	Is the final report neatly formatted with all the elements required for a technical Report?	Yes
2.	Is the Cover page in proper format as given in Annexure A?	Yes
3.	Is the Title page (Inner cover page) in proper format?	Yes
4.	(a) Is the Certificate from the Supervisor in proper format? (b) Has it been signed by the Supervisor?	Yes / Yes
5.	Is the Abstract included in the report properly written within one page? Have the technical keywords been specified properly?	Yes / Yes
6.	Is the title of your report appropriate? The title should be adequately descriptive, precise and must reflect the scope of the actual work done. Uncommon abbreviations / Acronyms should not be used in the title.	Yes
7.	Have you included the List of abbreviations / Acronyms?	Yes
8.	Does the Report contain a summary of the literature survey?	Yes
9.	Does the Table of Contents include page numbers? Are the Pages numbered properly (Chapter 1 starts on Page 1)? Are the Figures numbered properly (Figure number and title at the bottom)? Are the Tables numbered properly (Table number and title at the top)? Are the Captions for the Figures and Tables proper? Are the Appendices numbered properly with appropriate titles?	Yes (all)
10.	Is the conclusion of the Report based on discussion of the work?	Yes
11.	Are References or Bibliography given at the end of the Report? Have the References been cited properly inside the text? Are all the references cited in the body of the report?	Yes (all)
12.	Is the report format and content according to the guidelines? (The report should not be a mere printout of a PowerPoint presentation, or a user manual. Source code of software need not be included in the report.)	Yes

Declaration by Student:

I certify that I have properly verified all the items in this checklist and ensure that the report is in proper format as specified in the course handout.

Place: Bangalore

Date: 09 May 2026



Signature of the Student:

Name: Aarya Nanndaann Singh M N

ID No.: 2024MT03013

For Supervisor's Use:



Signature of the Supervisor:

Name: Mr. Hanumanthu Indrakanti

Date: 09 May 2026